

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №2
по курсу «Алгоритмы и структуры данных»
Тема: Сортировка слиянием. Метод декомпозиции
Вариант 7 (Продвинутый уровень)

Выполнила:
Кулакова Елизавета Алексеевна
К3260

Проверил:
Харлунин А. А.

Санкт-Петербург
2025 г.

Содержание отчета

Содержание отчета.....	2
Задачи по варианту (базовый уровень).....	3
Задача №1. Сортировка слиянием.....	3
Переписанная сортировка слиянием.....	14
Задача №2. Сортировка слиянием+.....	22
Задача №9. Метод Штрассена для умножения матриц.....	30
Задачи на продвинутый уровень.....	44
Задача №4. Бинарный поиск.....	44
Задача №8. Умножение многочленов.....	55
Вывод.....	65

Задачи по варианту (базовый уровень)

Задача №1. Сортировка слиянием

1. Используя псевдокод процедур Merge и Merge-sort из презентации к Лекции 2 (страницы 6-7), напишите программу сортировки слиянием на Python и проверьте сортировку, создав несколько случайных массивов, подходящих под параметры:

- Формат входного файла (input.txt). В первой строке входного файла содержится число n ($1 \leq n \leq 2 \cdot 10^4$) — число элементов в массиве. Во второй строке находятся n различных целых чисел, по модулю не превосходящих 10^9 .

- Формат выходного файла (output.txt). Одна строка выходного файла с отсортированным массивом. Между любыми двумя числами должен стоять ровно один пробел.

- Ограничение по времени. 2сек.

- Ограничение по памяти. 256 мб.

2. Для проверки можно выбрать наихудший случай, когда сортируется массив размера 1000, 10^4 , 10^5 чисел порядка 10^9 , отсортированных в обратном порядке; наилучший, когда массив уже отсортирован, и средний. Сравните, например, с сортировкой вставкой на этих же данных.

3. Перепишите процедуру Merge так, чтобы в ней не использовались сигнальные значения. Сигналом к остановке должен служить тот факт, что все элементы массива L или R скопированы обратно в массив A, после чего в этот массив копируются элементы, оставшиеся в непустом массиве.

4. или перепишите процедуру Merge (и, соответственно, Merge-sort) так, чтобы в ней не использовались значения границ и середины - p, r и q.

Листинг кода

```
from measurements import *
from random_array import *
from numpy import inf

@measure_performance
def get_data():
    with open('input.txt') as f:
        data = f.readlines()
```

```

return int(data[0]), list(map(int, data[1].split()))

def merge(arr, p, q, r):
    n1 = q - p
    n2 = r - q

    lft = [0 for _ in range(n1 + 1)]
    rght = [0 for _ in range(n2 + 1)]

    # lft[:n1] = arr[p:q]
    # rght[:n2] = arr[q:r]

    for i in range(n1):
        lft[i] = arr[p+i]
    for j in range(n2):
        rght[j] = arr[q+j]

    lft[-1] = inf
    rght[-1] = inf
    i, j = 0, 0

    for k in range(p, r):
        if lft[i] <= rght[j]:
            arr[k] = lft[i]
            i += 1
        else:
            arr[k] = rght[j]
            j += 1

    return arr

def merge_sort(arr, p, r):
    if p < r - 1:
        q = (p+r)//2
        merge_sort(arr, p, q)
        merge_sort(arr, q, r)
        return merge(arr, p, q, r)
    return arr

```

```

if __name__ == "__main__":
    generate_array(4, pattern='desc')
    ln, array = get_data()
    sarr = merge_sort(array, 0, ln)
    with open('output.txt', 'w') as fl:
        fl.write(" ".join(map(str, sarr)))
    assert sorted(array) == sarr, 'not sorted.....'

```

```

from measurments import *
from random_array import *
from numpy import inf

```

```

@measure_performance

```

```

def get_data():
    with open('input.txt') as f:
        data = f.readlines()

```

```

    return int(data[0]), list(map(int, data[1].split()))

```

```

def merge(arr, p, q, r):

```

```

    n1 = q - p
    n2 = r - q

```

```

    lft = [0 for _ in range(n1 + 1)]
    right = [0 for _ in range(n2 + 1)]

```

```

    # lft[:n1] = arr[:q]
    # right[:n2] = arr[q:]

```

```

    for i in range(n1):
        lft[i] = arr[p+i]
    for j in range(n2):
        right[j] = arr[q+j]

```

```

    lft[-1] = inf
    right[-1] = inf
    i, j = 0, 0

```

```

    for k in range(p, r):

```

```

        if lft[i] <= rght[j]:
            arr[k] = lft[i]
            i += 1
        else:
            arr[k] = rght[j]
            j += 1

    return arr

def merge_sort(arr, p, r):
    if p < r - 1:
        q = (p+r)//2
        merge_sort(arr, p, q)
        merge_sort(arr, q, r)
        return merge(arr, p, q, r)
    return arr

if __name__ == "__main__":
    generate_array(4, pattern='desc')
    ln, array = get_data()
    sarr = merge_sort(array, 0, ln)
    with open('output.txt', 'w') as fl:
        fl.write(" ".join(map(str, sarr)))
    assert sorted(array) == sarr, 'not sorted.....'

```

Листинг кода функции для формирования массива

```

import random
import numpy as np

def generate_array(n=None, pattern=None):
    if not n:
        n = random.randint(1, 2 * 10**4)
    if pattern == 'asc':
        fin_arr = sorted(np.random.randint(-10 ** 9, 10 ** 9, size=n))
    elif pattern == 'desc':
        fin_arr = sorted(np.random.randint(10 ** 9, 10 ** 10, size=n),
reverse=True)
    else:

```

```

    fin_arr = np.random.randint(-10 ** 9, 10 ** 9, size=n)

    with open('input.txt', 'w') as f:
        f.writelines([f'{str(n)}\n', " ".join(map(str, fin_arr))])

    return None

```

```

import random
import numpy as np

def generate_array(n=None, pattern=None):
    if not n:
        n = random.randint(1, 2 * 10**4)
    if pattern == 'asc':
        fin_arr = sorted(np.random.randint(-10 ** 9, 10 ** 9, size=n))
    elif pattern == 'desc':
        fin_arr = sorted(np.random.randint(10 ** 9, 10 ** 10, size=n),
reverse=True)
    else:
        fin_arr = np.random.randint(-10 ** 9, 10 ** 9, size=n)

    with open('input.txt', 'w') as f:
        f.writelines([f'{str(n)}\n', " ".join(map(str, fin_arr))])

    return None

```

Текстовое объяснение решения.

Для выполнения первого задания были написаны функции `merge_sort()` и `merge()`, а также вспомогательные функции `get_data()`, которая не принимает аргументов, но считывает данные из файла `input.txt` по заданному формату и возвращает длину массива и сам массив, и `generate_array()` для формирования массива (отсортированного по убыванию или возрастанию или не отсортированного вовсе). Функция `merge_sort()` возвращает отсортированный массив чисел.

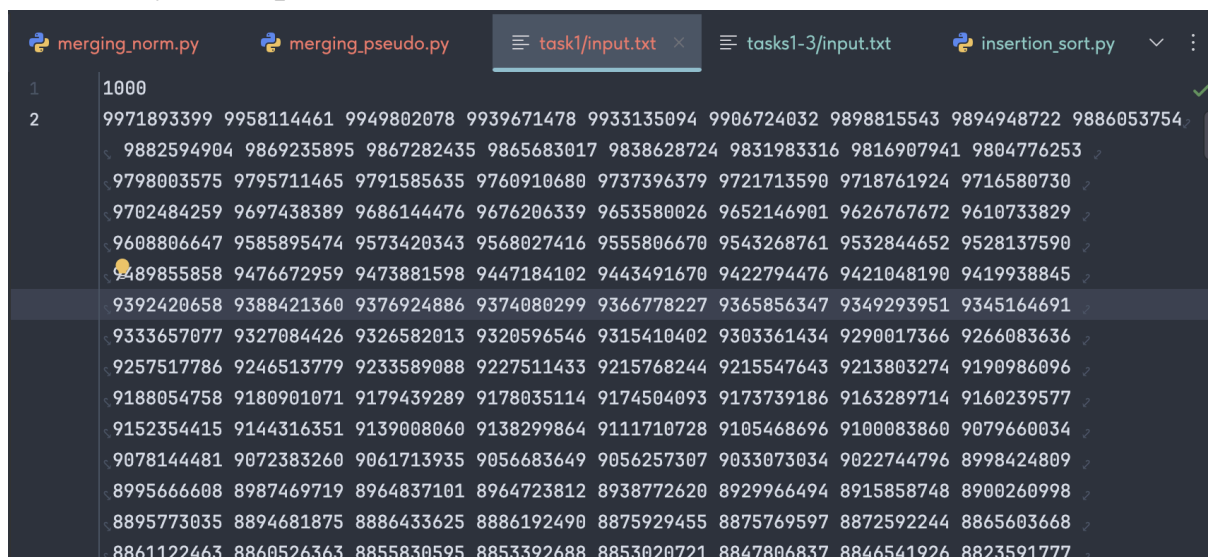
В процессе работы функции `merge_sort()` массив “делится” на две части, каждая из которых рекурсивно сортируется. Когда части разделились на массивы длиной 1, происходит слияние: сравниваются элементы массивов, начиная с индекса 0, и записываются в оригинальный массив под новыми (или нет) индексами по результатам сортировки. Далее

такой процесс производится уже для двух массивов одной части длиной 2 и так далее, глубина рекурсии зависит от длины массива. Слияние заканчивается, когда в обоих массивах достигнут элемент-заглушка `inf`. Таким образом на финальное слияние функции приходит массив с отсортированными половинами, и уже из этих половин формируется финальный отсортированный массив.

После сортировки массива в файл `output.txt` записывается строковое значение, образованное конкатенацией значений отсортированного массива. Также сам массив возвращается функцией.

Для генерации массива была написана функция `generate_array()`, которая в качестве аргумента может принимать целое число – размерность массива – и паттерн построения массива (в убывающем, возрастающем или хаотичном порядке). Если число при вызове не передается, то генерируется число в заданном диапазоне, а затем на его основании генерируется массив, вся информация записывается в файл `input.txt`.

Результат работы кода:



```
1 1000
2 9971893399 9958114461 9949802078 9939671478 9933135094 9906724032 9898815543 9894948722 9886053754
  9882594904 9869235895 9867282435 9865683017 9838628724 9831983316 9816907941 9804776253
  9798003575 9795711465 9791585635 9760910680 9737396379 9721713590 9718761924 9716580730
  9702484259 9697438389 9686144476 9676206339 9653580026 9652146901 9626767672 9610733829
  9608806647 9585895474 9573420343 9568027416 9555806670 9543268761 9532844652 9528137590
  9489855858 9476672959 9473881598 9447184102 9443491670 9422794476 9421048190 9419938845
  9392420658 9388421360 9376924886 9374080299 9366778227 9365856347 9349293951 9345164691
  9333657077 9327084426 9326582013 9320596546 9315410402 9303361434 9290017366 9266083636
  9257517786 9246513779 9233589088 9227511433 9215768244 9215547643 9213803274 9190986096
  9188054758 9180901071 9179439289 9178035114 9174504093 9173739186 9163289714 9160239577
  9152354415 9144316351 9139008060 9138299864 9111710728 9105468696 9100083860 9079660034
  9078144481 9072383260 9061713935 9056683649 9056257307 9033073034 9022744796 8998424809
  8995666608 8987469719 8964837101 8964723812 8938772620 8929966494 8915858748 8900260998
  8895773035 8894681875 8886433625 8886192490 8875929455 8875769597 8872592244 8865603668
  8861122463 8860526363 8855830595 8853392688 8853020721 8847806837 8846541926 8823591777
```

Рисунок 1 – Файл `input.txt` с входными данными худший случай (убывание и элементы порядка 10^9 , 1000 элементов)

```
cs1-3/input.txt  insertion_sort.py  tasks1-3/random_array.py  output.txt  task1/random_array.py  ⌵  ⋮

1  1009634356 1018585860 1020725849 1020938620 1057032122 1063388678 1073524877 1085271328 1090108515 ✓
  1100502955 1112135210 1128156318 1128344239 1132114153 1135725935 1144981399 1152994310 ⌵
  1153302051 1156899547 1159320729 1170543528 1174133706 1185441505 1205644610 1228597025 ⌵
  1249175497 1257598622 1261349923 1263017181 1265634978 1273583465 1285340728 1293162349 ⌵
  1293910389 1304413099 1308079854 1313090837 1315546652 1316219743 1318968334 1327043092 ⌵
  1331758074 1351444753 1367363519 1375301651 1376361767 1387142405 1392246408 1397882659 ⌵
  1406942302 1407301444 1407564688 1415026007 1419894936 1428459104 1448110259 1484835676 ⌵
  1486096127 1488769992 1496998785 1522298166 1527578464 1536083403 1592943514 1604821197 ⌵
  1606761564 1622376216 1627850533 1629257110 1639558757 1657649974 1660772695 1677983531 ⌵
  1680653695 1681054813 1682811494 1697652268 1733121177 1745027031 1746596070 1748428574 ⌵
  1750771194 1752299179 1791338197 1802401236 1810615867 1822349030 1832872635 1843382273 ⌵
  1849264474 1864186313 1874392831 1897963220 1900433013 1910140262 1915752294 1927122800 ⌵
  1935155473 1935832369 1951571631 1954745470 1956863722 1985215314 1999146041 2018459202 ⌵
  2026859293 2036117078 2047409411 2058320118 2067948353 2088001521 2109558562 2116319093 ⌵
  2123601354 2130737308 2143328782 2152771222 2154156445 2161228071 2167244610 2189241753 |
  2192883089 2211029883 2229851390 2261936712 2263179591 2267383810 2289614160 2295687561 ⌵
```

Рисунок 2 – Результат работы функции, худший случай (убывание и элементы порядка 10^9 , 1000 элементов)

```
measurments.py  task1/input.txt  tasks1-3/input.txt  insertion_sort.py  tasks1-3/random_arr  ⌵  ⋮

1  10000 ✓
2  9999852985 9999487076 9997077698 9996793364 9995054911 9994442702 9993308227 9991336637 9989750604 ⌵
  9988072249 9987852097 9987700125 9987214233 9986095814 9984827717 9982742501 9982676526 ⌵
  9981284614 9981085805 9977639327 9975803006 9975475608 9975069342 9972404801 9972078361 ⌵
  9971527003 9970362720 9969711633 9966488971 9964940158 9962841004 9961256035 9959920246 ⌵
  9959676007 9958652185 9957887516 9957643566 9957433482 9955441324 9953787461 9952672127 ⌵
  9952479313 9952096056 9949229916 9949199923 9949051890 9946997424 9946172156 9945817047 ⌵
  9944903489 9944816588 9944715210 9942332078 9941428400 9939514385 9939418079 9938385075 ⌵
  9936792402 9936674895 9936321925 9933119453 9932015420 9929957709 9929681465 9929466624 ⌵
  9929094351 9928249427 9925669265 9924871398 9920825403 9918834887 9918809650 9916845580 ⌵
  9916459861 9916150702 9916080726 9914869234 9914865744 9913896835 9913600384 9912250122 ⌵
  9911383718 9910249390 9909523444 9909186001 9909009553 9907238856 9905379321 9904873613 ⌵
  9904278597 9903924207 9902328644 9902255731 9902131239 9901947212 9901045565 9900565277 ⌵
  9900562172 9899878775 9898852722 9898315094 9896776209 9895596163 9895558050 9894934164 ⌵
  9894144956 9892835597 9892214092 9892110770 9891591758 9890815990 9889857862 9885254587 ⌵
  9883856660 9883319097 9883303393 9883154505 9878846689 9876966508 9876463889 9875465389 ⌵
```

Рисунок 3 – Файл input.txt с входными данными худший случай (убывание и элементы порядка 10^9 , 10000 элементов)

```
task1/input.txt  tasks1-3/input.txt  insertion_sort.py  tasks1-3/random_array.py  output.txt  ⌵  ⋮

1  1000763636 1000806920 1001297777 1004760654 1005107757 1006250425 1006348090 1006791017 1006803375 ✓
  1007039284 1007870031 1007985875 1008191674 1008346571 1008379342 1008415216 1009197856 ⌵
  1009659653 1010851212 1011721309 1012820480 1012888912 1013127593 1013895064 1014272530 ⌵
  1014708634 1015337738 1015379039 1015555995 1015754035 1016114695 1018342961 1018373718 ⌵
  1019778845 1021404732 1021436939 1021531970 1022221384 1023364216 1024441792 1025185350 ⌵
  1026636590 1027674622 1027821780 1029898016 1029976324 1032985800 1033284552 1033427205 ⌵
  1033793142 1036388729 1036704271 1036796002 1036970420 1037018654 1038773712 1039082449 ⌵
  1039748941 1040348520 1040720531 1040978731 1041286802 1041647097 1043638112 1044026437 ⌵
  1044122049 1044231628 1046400806 1046497701 1046643312 1046828214 1047294547 1049089677 ⌵
  1051235902 1051344659 1052411218 1052683286 1052928172 1053672723 1053977431 1053982839 ⌵
  1055569524 1056089694 1056444080 1056822667 1057292609 1058415106 1065093214 1065389590 ⌵
  1066267297 1066955908 1066981889 1067406231 1068340798 1069355989 1070194468 1070958096 ⌵
  1072554592 1072941540 1074332679 1074354042 1074931453 1076007735 1076319088 1076680206 ⌵
  1076904965 1078516729 1078529663 1078773425 1079493610 1079572467 1082886330 1084440830 ⌵
  1084472301 1084772528 1086690077 1087088459 1088044259 1088256062 1088476365 1088744879 ⌵
  1089875780 1090280418 1090838749 1092184359 1092482296 1093239016 1093542084 1094509128 ⌵
```

Рисунок 4 – Результат работы функции, худший случай (убывание и элементы порядка 10^9 , 10000 элементов)

```

merging_pseudo.py task1/input.txt tasks1-3/input.txt insertion_sort.py tasks1-3/random_c
1 100000
2 9999910138 9999736857 9999722914 9999458584 9999167440 9999153863 9999144166 9999112371 9999088357
  9999049657 9999004279 9998958745 9998915220 9998841660 9998712628 9998642916 9998411805
  9998393008 9998386470 9998333460 9998319962 9998278626 9998276294 9998192894 9998174369
  9998139838 9998057712 9997975145 9997924979 9997920979 9997876703 9997852176 9997679183
  9997653230 9997531363 9997477362 9997178662 9997171212 9997150771 9996974277 9996853779
  9996777201 9996763225 9996758846 9996752098 9996718196 9996635109 9996472209 9996392142
  9996284275 9996138591 9995901937 9995820963 9995813645 9995743389 9995688483 9995470853
  9995468796 9995415894 9995223026 9995046069 9995013648 9994966594 9994850216 9994680903
  9994625450 9994571145 9994344238 9994214207 9993795062 9993735846 9993558548 9993239206
  9993092677 9993082013 9992888855 9992843118 9992822573 9992729472 9992721225 9992458989
  9992381759 9992357146 9992334146 9992106094 9991900275 9991819266 9991698197 9991650000
  9991536460 9991378063 9991342369 9991282453 9991246784 9991155031 9991139386 9991033541
  9990984261 9990936379 9990795923 9990792221 9990774066 9990744775 9990675398 9990628188
  9990596539 9990570208 9990510041 9990479861 9990378317 9990283545 9990196735 9990150290
  9990106561 9990058621 9990041336 9990035292 9990014914 9990011354 9989717250 9989673239

```

Рисунок 5 – Файл input.txt с входными данными худший случай (убывание и элементы порядка 10^9 , 100000 элементов)

```

-3/input.txt insertion_sort.py tasks1-3/random_array.py output.txt task1/random_array.py
1 1000077528 1000139450 1000246801 1000259924 1000266707 1000372047 1000383337 1000515699 1000670917
  1000743497 1000791884 1000932758 1001149453 1001186628 1001212959 1001382070 1001418516
  1001439179 1001456212 1001463221 1001532928 1001578027 1001935396 1001981970 1002023204
  1002147536 1002718922 1002794553 1002819085 1002886722 1002923172 1002940195 1003008103
  1003209710 1003304894 1003348309 1003413903 1003505929 1003565006 1003833159 1003960575
  1004051509 1004074302 1004118804 1004313784 1004320590 1004539712 1004632650 1004714225
  1004746333 1004799839 1004906723 1004986572 1005108113 1005272302 1005319658 1005349333
  1005457562 1005779299 1005793076 1005981774 1006021402 1006361057 1006399702 1006693660
  1006706938 1006716308 1006822932 1007037131 1007071366 1007093729 1007111745 1007143633
  1007495493 1007515982 1007620778 1007760291 1007953894 1008077977 1008144197 1008228262
  1008285719 1008399161 1008423860 1008730516 1008921317 1009063089 1009077066 1009267931
  1009369861 1009470113 1009486002 1009519312 1009535346 1009553461 1009604516 1009714353
  1009760130 1009766654 1009772979 1009858435 1009886460 1009918507 1010028137 1010090303
  1010107708 1010134594 1010136153 1010201111 1010246061 1010271660 1010380928 1010546405
  1010851509 1010969450 1011018582 1011050451 1011068670 1011086993 1011150568 1011401678
  1011599802 1011638105 1011666921 1011730711 1011744988 1011884500 1011921475 1012012284

```

Рисунок 6 – Результат работы функции, худший случай (убывание и элементы порядка 10^9 , 100000 элементов)

```
merging_pseudo.py task1/input.txt tasks1-3/input.txt insertion_sort.py tasks1-3/random_c
1 10638
2 75622754 -514335709 -132344808 967597597 -855596329 -311750197 301769205 909645202 74330353
   -487789488 933565591 -550805595 641199521 6616280 -830705931 -397573104 46940946 -64917783
   -609715524 -188415554 41081557 -953585572 566106387 601146853 871328587 -196692190 991134689
   -277441907 -502079323 369110458 975019976 -323773829 -396713440 -328105132 -410793519 208277588
   986443415 161484043 69491494 -385638583 -127471592 951430615 -130735249 -76125409 -968474794
   -38775211 765098250 -35435327 -572579336 -308490836 -934262332 -531260451 253371858 -990801827
   -919089181 -269693698 663191369 231371637 -349843111 134010674 844574030 167166100 383298981
   -928638327 -227153996 912366696 175525924 -948147158 -519662424 587520190 768129816 -851490288
   605697313 770992615 60733668 7710588 -987764987 -746045920 313372426 519654198 390418971
   -130629114 -620653188 -314318921 795988610 930288554 -967592866 -316696628 -278652321 142255668
   733364587 637552481 505148118 -804410033 -35410434 510820587 361426817 286439549 140209503
   -454523971 164645561 -951136122 -875911662 -627574615 -798239939 -344649511 -954149770 -938954451
   874579354 -469876310 720356772 992913988 -251698179 508281642 -236110079 470951139 279213360
   -165415782 38878187 356608327 -57764633 843600320 -856115286 78027581 -509960651 -167814579
   -153969885 -707648510 -5358798 420602753 864689961 -670984098 231033237 -147097714 -392325786
```

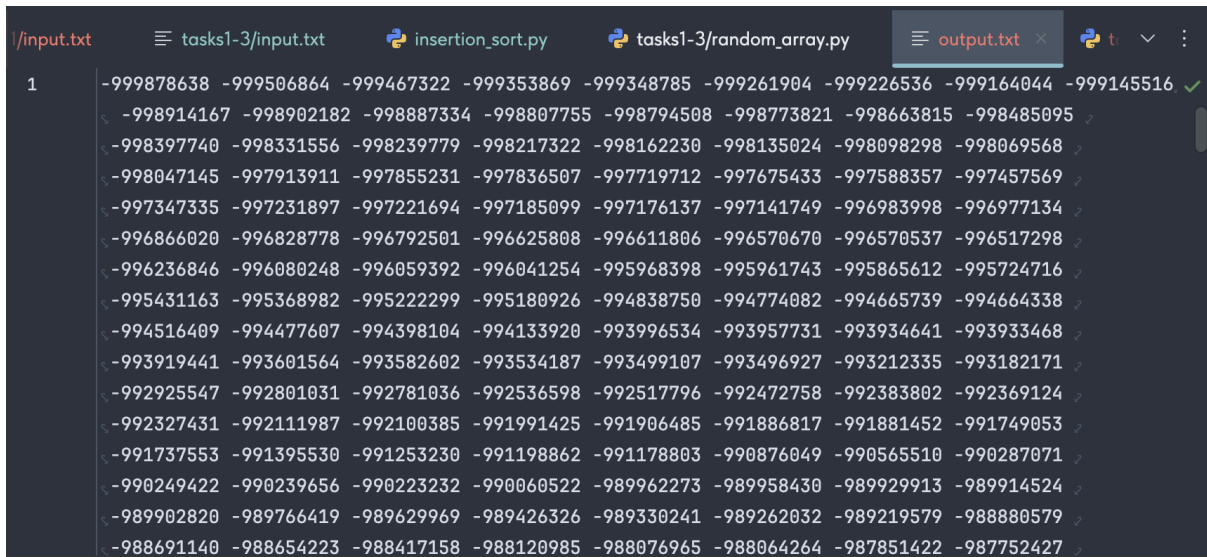
Рисунок 7 – Файл input.txt с входными данными средний случай

```
tasks1-3/input.txt insertion_sort.py tasks1-3/random_array.py output.txt task1/random_c
1 -999415594 -999149647 -999062139 -999022416 -998200318 -997529159 -997088550 -996981616 -996685820
   -996401964 -995774710 -995610298 -995347398 -995191905 -994831840 -994521544 -994024160
   -993938703 -992900823 -992824275 -992789125 -992515859 -992468822 -992240514 -991989141
   -991842637 -991750857 -991669373 -991544688 -991488809 -991442234 -991372283 -990976553
   -990890010 -990838499 -990810964 -990801827 -990795555 -990697450 -990519077 -990287163
   -990043086 -989798404 -989684954 -989514201 -989467295 -989423977 -989260103 -989254340
   -988633913 -988487913 -988386412 -988238923 -988044451 -987916463 -987779967 -987764987
   -987762179 -987413793 -987328098 -986794160 -986721112 -986511413 -986277958 -986212215
   -985998085 -985831111 -985716571 -985603669 -985194085 -985167189 -985006668 -984776479
   -984641798 -984478619 -984353414 -984176196 -984075448 -983847226 -983035451 -982820234
   -982758339 -982598469 -982266299 -981956197 -981808967 -981754999 -981707932 -981673340
   -981284928 -981234406 -981142888 -980641601 -980595423 -979908799 -979617799 -979602404
   -979560509 -979296280 -979118437 -978931735 -978782580 -978602161 -978537768 -978404199
   -978315246 -978031513 -978009399 -977939444 -977682597 -977596908 -977551558 -977378249
   -977369795 -977184866 -977173726 -977144752 -976810064 -976789773 -976728873 -976668165
   -976461694 -975929180 -975716432 -975637202 -975585469 -975394755 -975067535 -974832854
```

Рисунок 8 – Результат работы функции, средний случай

```
g_norm.py merging_pseudo.py task1/input.txt tasks1-3/input.txt insertion_sort.py tas
1 20000
2 -999878638 -999506864 -999467322 -999353869 -999348785 -999261904 -999226536 -999164044 -999145516
   -998914167 -998902182 -998887334 -998807755 -998794508 -998773821 -998663815 -998485095
   -998397740 -998331556 -998239779 -998217322 -998162230 -998135024 -998098298 -998069568
   -998047145 -997913911 -997855231 -997836507 -997719712 -997675433 -997588357 -997457569
   -997347335 -997231897 -997221694 -997185099 -997176137 -997141749 -996983998 -996977134
   -996866020 -996828778 -996792501 -996625808 -996611806 -996570670 -996570537 -996517298
   -996236846 -996080248 -996059392 -996041254 -995968398 -995961743 -995865612 -995724716
   -995431163 -995368982 -995222299 -995180926 -994838750 -994774082 -994665739 -994664338
   -994516409 -994477607 -994398104 -994133920 -993996534 -993957731 -993934641 -993933468
   -993919441 -993601564 -993582602 -993534187 -993499107 -993496927 -993212335 -993182171
   -992925547 -992801031 -992781036 -992536598 -992517796 -992472758 -992383802 -992369124
   -992327431 -992111987 -992100385 -991991425 -991906485 -991886817 -991881452 -991749053
   -991737553 -991395530 -991253230 -991198862 -991178803 -990876049 -990565510 -990287071
   -990249422 -990239656 -990223232 -990060522 -989962273 -989958430 -989929913 -989914524
   -989902820 -989766419 -989629969 -989426326 -989330241 -989262032 -989219579 -988880579
```

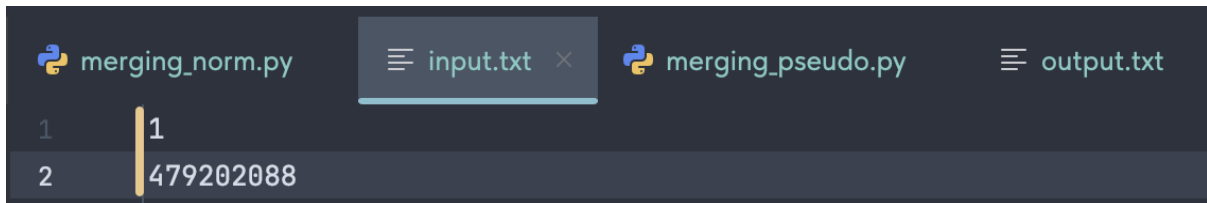
Рисунок 9 – Файл input.txt с входными данными лучший случай



```
1 -999878638 -999506864 -999467322 -999353869 -999348785 -999261904 -999226536 -999164044 -999145516 ✓
2 -998914167 -998902182 -998887334 -998807755 -998794508 -998773821 -998663815 -998485095
3 -998397740 -998331556 -998239779 -998217322 -998162230 -998135024 -998098298 -998069568
4 -998047145 -997913911 -997855231 -997836507 -997719712 -997675433 -997588357 -997457569
5 -997347335 -997231897 -997221694 -997185099 -997176137 -997141749 -996983998 -996977134
6 -996866020 -996828778 -996792501 -996625808 -996611806 -996570670 -996570537 -996517298
7 -996236846 -996080248 -996059392 -996041254 -995968398 -995961743 -995865612 -995724716
8 -995431163 -995368982 -995222299 -995180926 -994838750 -994774082 -994665739 -994664338
9 -994516409 -994477607 -994398104 -994133920 -993996534 -993957731 -993934641 -993933468
10 -993919441 -993601564 -993582602 -993534187 -993499107 -993496927 -993212335 -993182171
11 -992925547 -992801031 -992781036 -992536598 -992517796 -992472758 -992383802 -992369124
12 -992327431 -992111987 -992100385 -991991425 -991906485 -991886817 -991881452 -991749053
13 -991737553 -991395530 -991253230 -991198862 -991178803 -990876049 -990565510 -990287071
14 -990249422 -990239656 -990223232 -990060522 -989962273 -989958430 -989929913 -989914524
15 -989902820 -989766419 -989629969 -989426326 -989330241 -989262032 -989219579 -988880579
16 -988691140 -988654223 -988417158 -988120985 -988076965 -988064264 -987851422 -987752427
```

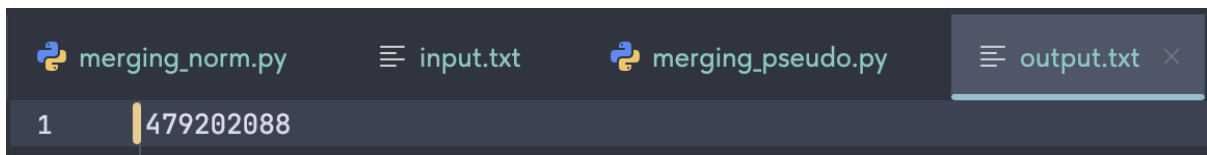
Рисунок 10 – Результат работы функции, лучший случай

Результат работы кода на максимальных и минимальных значениях:



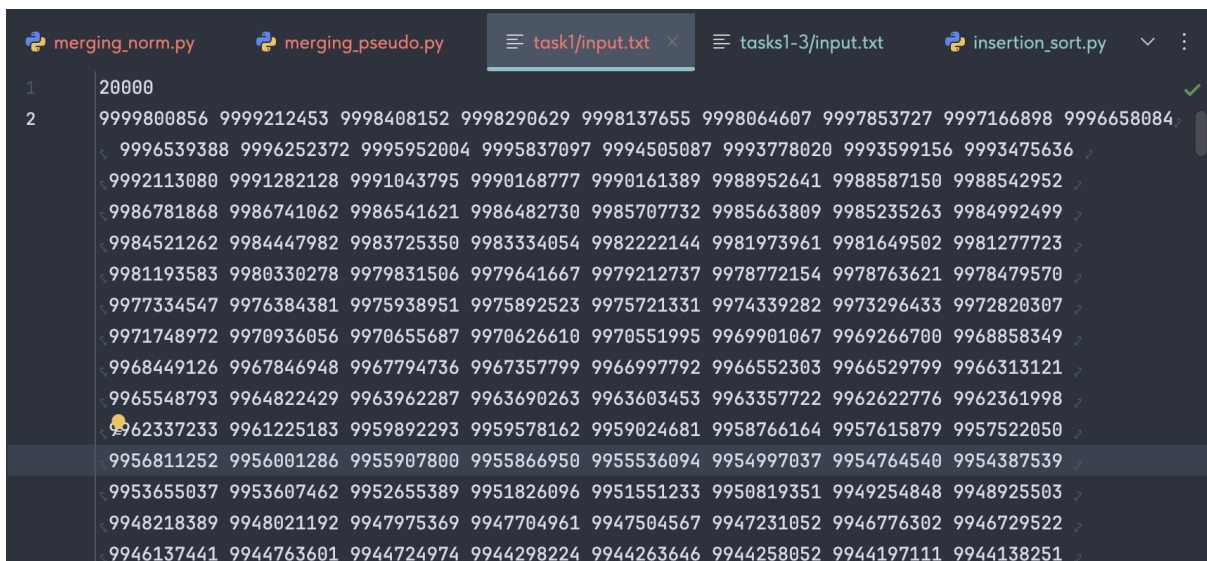
```
1 1
2 479202088
```

Рисунок 11 – Файл input.txt с вводными данными – минимальными значениями



```
1 479202088
```

Рисунок 12 – Файл output.txt после выполнения функции с минимальными значениями



```
1 20000 ✓
2 9999800856 9999212453 9998408152 9998290629 9998137655 9998064607 9997853727 9997166898 9996658084
3 9996539388 9996252372 9995952004 9995837097 9994505087 9993778020 9993599156 9993475636
4 9992113080 9991282128 9991043795 9990168777 9990161389 9988952641 9988587150 9988542952
5 9986781868 9986741062 9986541621 9986482730 9985707732 9985663809 9985235263 9984992499
6 9984521262 9984447982 9983725350 9983334054 9982222144 9981973961 9981649502 9981277723
7 9981193583 9980330278 9979831506 9979641667 9979212737 9978772154 9978763621 9978479570
8 9977334547 9976384381 9975938951 9975892523 9975721331 9974339282 9973296433 9972820307
9 9971748972 9970936056 9970655687 9970626610 9970551995 9969901067 9969266700 9968858349
10 9968449126 9967846948 9967794736 9967357799 9966997792 9966552303 9966529799 9966313121
11 9965548793 9964822429 9963962287 9963690263 9963603453 9963357722 9962622776 9962361998
12 9962337233 9961225183 9959892293 9959578162 9959024681 9958766164 9957615879 9957522050
13 9956811252 9956001286 9955907800 9955866950 9955536094 9954997037 9954764540 9954387539
14 9953655037 9953607462 9952655389 9951826096 9951551233 9950819351 9949254848 9948925503
15 9948218389 9948021192 9947975369 9947704961 9947504567 9947231052 9946776302 9946729522
16 9946137441 9944763601 9944724974 9944298224 9944263646 9944258052 9944197111 9944138251
```

Рисунок 13 – Файл input.txt с вводными данными – максимальными значениями

```

1 1000153383 1000479923 1000788744 1000963346 1001082146 1001432524 1001507065 1002268107 1002606577
  1002886411 1003231294 1003274592 1004349599 1004355855 1004963870 1005200932 1005565657
  1006000438 1006638235 1007093189 1007117644 1007175978 1008005135 1008324000 1009534489
  1009710839 1010570820 1010835706 1012462474 1012803178 1012858042 1013607430 1013755425
  1013790060 1013931402 1015085022 1015119085 1015249737 1015758228 1015857163 1016531861
  1017327101 1017647922 1017763139 1017936173 1018034121 1018141615 1018675965 1018698020
  1018907350 1020761343 1020908757 1020995592 1021521063 1021960430 1022044582 1022280582
  1022874445 1023401477 1023460987 1023503717 1024369864 1025294294 1025742722 1025762762
  1027312432 1027358697 1028122267 1028179260 1028315846 1028391928 1028654631 1028730919
  1029254433 1029323425 1029549288 1029846789 1029951964 1030151149 1030387987 1030668162
  1030725294 1030859232 1030897948 1032143021 1033027539 1034806641 1035038853 1036405538
  1036583229 1036593356 1036817402 1037608105 1038404226 1038515488 1038556142 1038614648
  1038756925 1039332269 1041051113 1041809654 1042547187 1044938357 1045278989 1045500079
  1046138955 1048743055 1049474860 1049547864 1049568919 1049934571 1050441030 1050541640
  1051124674 1051190484 1051853868 1051916186 1053374490 1054723496 1055015963 1056456866
  1056867405 1056987640 1057491435 1058979103 1059787790 1059907530 1060090716 1060513818
  
```

Рисунок 14 – Файл output.txt после выполнения функции с максимальными значениями

Таблица затрат памяти и времени по заданию 1:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0004 сек	13.74 KB
Худший случай (убывание и элементы порядка 10^9 , 1000 элементов)	0.0016 сек	110.46 KB
Худший случай (убывание и элементы порядка 10^9 , 10000 элементов)	0.0102 сек	1085.85 KB
Худший случай (убывание и элементы порядка 10^9 , 100000 элементов)	0.2202 сек	10746.88 KB
Средний случай	0.0115 сек	1090.22 KB
Лучший случай (отсортированный массив)	0.0203 сек	2073.23 KB
Верхняя граница диапазона значений входных данных из текста задачи	0.0200 сек	2175.50 KB

Таблица сравнения затрат времени и памяти для сортировки слиянием и сортировки вставкой

	Сортировка слиянием		Сортировка вставкой	
	Время выполнения	Затраты памяти	Время выполнения	Затраты памяти
Худший случай (убывание и элементы порядка 10^9 , 1000 элементов)	0.0016 сек	110.46 KB	0.8809 сек	40.0 KB
Худший случай (убывание и элементы порядка 10^9 , 10000 элементов)	0.0102 сек	1085.85 KB	144.3221 сек	396 KB
Худший случай (убывание и элементы порядка 10^9 , 100000 элементов)	0.2202 сек	10746.88 KB	-	-
Средний случай	0.0115 сек	1090.22 KB	87.2658 сек	374 KB
Лучший случай (отсортированный массив)	0.0203 сек	2073.23 KB	0.1630 сек	716 KB

Переписанная сортировка слиянием

Листинг кода

```
from measurments import *
from random_array import *

@measure_performance
def get_data():
    with open('input.txt') as f:
        data = f.readlines()
```

```

return int(data[0]), list(map(int, data[1].split()))

def merge_sort(n, arr):
    if n == 1:
        return arr
    l_arr = arr[:n//2]
    r_arr = arr[n//2:]
    l_sorted = merge_sort(len(l_arr), l_arr)
    r_sorted = merge_sort(len(r_arr), r_arr)

    i, j = 0, 0
    for k in range(n):
        if l_sorted[i] <= r_sorted[j]:
            arr[k] = l_sorted[i]
            if i == len(l_sorted) - 1:
                arr[k+1:] = r_sorted[j:]
                break
            i += 1
        else:
            arr[k] = r_sorted[j]
            if j == len(r_sorted) - 1:
                arr[k+1:] = l_sorted[i:]
                break
            j += 1
    return arr

if __name__ == "__main__":
    generate_array()
    ln, array = get_data()
    sarr = merge_sort(ln, array)
    with open('output.txt', 'w') as fl:
        fl.write(" ".join(map(str, sarr)))
    assert sorted(array) == sarr, 'not sorted.....'

```

```

from measurments import *
from random_array import *

@measure_performance

```

```

def get_data():
    with open('input.txt') as f:
        data = f.readlines()

    return int(data[0]), list(map(int, data[1].split()))

def merge_sort(n, arr):
    if n == 1:
        return arr
    l_arr = arr[:n//2]
    r_arr = arr[n//2:]
    l_sorted = merge_sort(len(l_arr), l_arr)
    r_sorted = merge_sort(len(r_arr), r_arr)

    i, j = 0, 0
    for k in range(n):
        if l_sorted[i] <= r_sorted[j]:
            arr[k] = l_sorted[i]
            if i == len(l_sorted) - 1:
                arr[k+1:] = r_sorted[j:]
                break
            i += 1
        else:
            arr[k] = r_sorted[j]
            if j == len(r_sorted) - 1:
                arr[k+1:] = l_sorted[i:]
                break
            j += 1
    return arr

if __name__ == "__main__":
    generate_array(pattern='desc')
    ln, array = get_data()
    sarr = merge_sort(ln, array)
    with open('output.txt', 'w') as fl:
        fl.write(" ".join(map(str, sarr)))
    assert sorted(array) == sarr, 'not sorted.....'

```

Текстовое объяснение решения

В этом алгоритме слияние и сортировка были объединены в одну функцию `merge_sort()`.

На вход функция принимает длину массива и сам массив. Если длина массива равна 1, то функция возвращает его. В ином случае массив делится на левый и правый подмассивы по среднему элементу, эти массивы так же обрабатываются функцией `merge_sort()` рекурсивно.

Когда все возможные подмассивы получены (длиной 1), начинается слияние. Начиная с 0 индекса подмассивов, сравниваются элементы, меньший записывается в оригинальный массив. Сравнение продолжается, пока элементы в одном из массивов не закончатся. Когда элементы в одном из массивов исчерпаны, оставшаяся часть незакончившегося массива записывается в оригинальный массив. Функция возвращает отсортированный массив и приступает к слиянию подмассивов большего размера.

В блоке `if __name__ == "__main__"` вызываются все необходимые функции, а результат сортировки слиянием сравнивается с результатом встроенной в Python функции `sorted()`.

Результат работы кода:

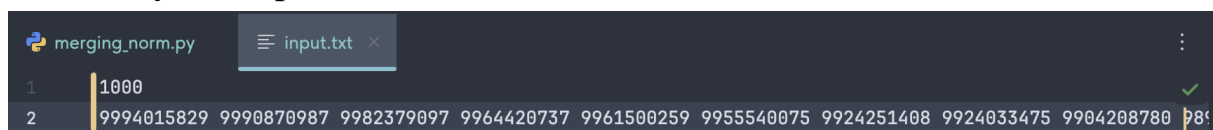


Рисунок 15 – Файл `input.txt` с входными данными худший случай (убывание и элементы порядка 10^9 , 1000 элементов)

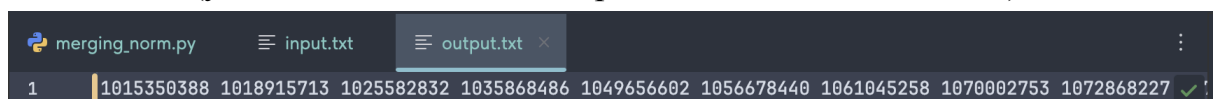


Рисунок 16 – Результат работы функции, худший случай (убывание и элементы порядка 10^9 , 1000 элементов)

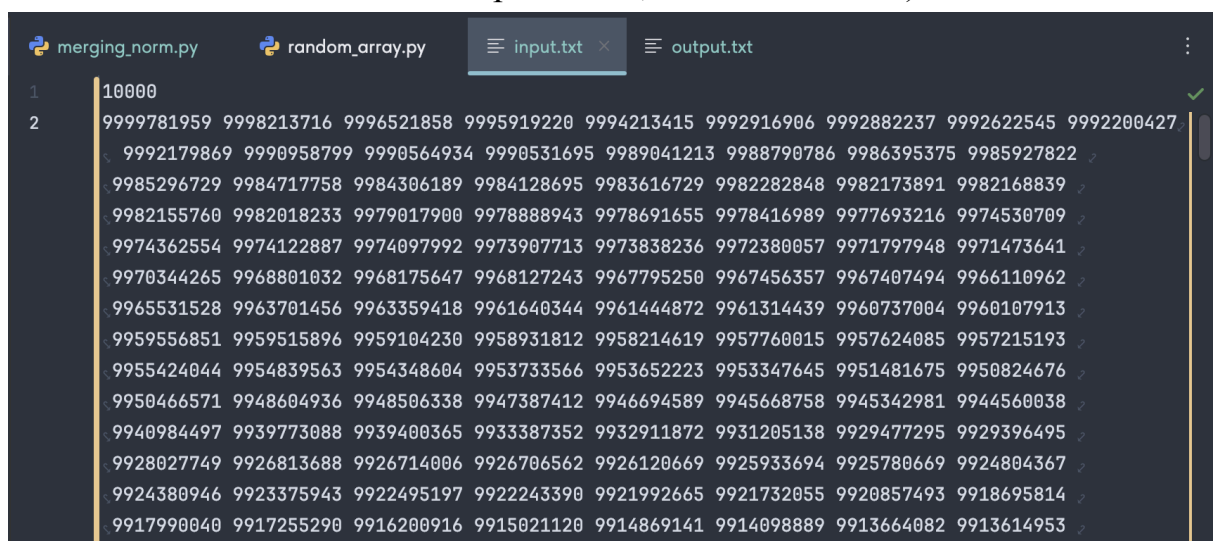


Рисунок 17 – Файл input.txt с входными данными худший случай
(убывание и элементы порядка 10^9 , 10000 элементов)

```

merging_norm.py random_array.py input.txt output.txt
1 1000513733 1000542191 1001603566 1002236641 1002736839 1004311418 1004721539 1004805336 1005119058
  1005630812 1006193485 1006809798 1007234921 1008658985 1010996387 1012365224 1012612780
  1012655640 1013062071 1014245818 1015624810 1015755504 1016734377 1017867635 1018366867
  1021987048 1022586894 1023653298 1024207601 1024918547 1025112095 1025380984 1025478235
  1026550580 1026877390 1027357250 1027531590 1027775228 1028865212 1029129220 1029296432
  1030075834 1030663835 1031079250 1033420290 1034224050 1034253670 1035956147 1037024165
  1037872812 1038026909 1039177758 1040532309 1042237222 1043131598 1043874105 1043963670
  1045811836 1047616093 1052725511 1055293986 1058360147 1058544767 1058732594 1062607542
  1063923504 1064732598 1065346746 1065592781 1066743496 1067818472 1068909722 1071007051
  1075232663 1076682110 1078419536 1079353747 1080195643 1080252700 1082065396 1085946817
  1086289563 1086964502 1087377811 1089227940 1089893231 1090930461 1092467910 1095241259
  1095529595 1095989552 1097320218 1097717279 1099178424 1099926102 1100721239 1102842130
  1102856436 1103131454 1105108149 1106613561 1108137091 1109054502 1110564393 1111189412
  1111362627 1115316949 1115587901 1115616932 1116647625 1116998693 1117286495 1118218452
  1118830274 1119021059 1120361513 1120412381 1120527635 1120791526 1121329015 1122650530

```

Рисунок 18 – Результат работы функции, худший случай (убывание и
элементы порядка 10^9 , 10000 элементов)

```

merging_norm.py random_array.py input.txt output.txt
1 100000
2 9999993132 9999988046 99999898773 9999828472 9999720352 9999691621 9999675454 9999646375 9999569028
  9999380780 9999323686 9999234169 9999230761 9999158162 9999139024 9999050775 9998778992
  9998762002 9998441976 9998408884 9998354287 9998254741 9998135281 9998112074 9998078060
  9998061620 9997845042 9997791246 9997784268 9997782411 9997700539 9997401065 9997369488
  9997357255 9997352422 9997314559 9997293016 9997264510 9997243315 9997017295 9996946133
  9996794031 9996608914 9996315948 9996260224 9996188818 9996027195 9995851799 9995748932
  9995550934 9995339340 9995133150 9995026552 9994988553 9994862115 9994783852 9994514031
  9994446881 9994438418 9994418449 9994356249 9994309395 9994271461 9994122369 9994119323
  9994046238 9993667005 9993663627 9993540919 9993453807 9993270039 9993254206 9993047885
  9992991304 9992824642 9992725168 9992572243 9992567443 9992538471 9992379029 9992311275
  9992268561 9992183860 9992063257 9991923060 9991691284 9991415984 9991395510 9991344621
  9991224994 9991151563 9990961246 9990940297 9990899983 9990880249 9990798254 9990724483
  9990409982 9990358198 9990297835 9990207989 9990158012 9990135358 9990056843 9989970907
  9989750125 9989691983 9989688999 9989506276 9989325115 9989289484 9989248158 9989239401

```

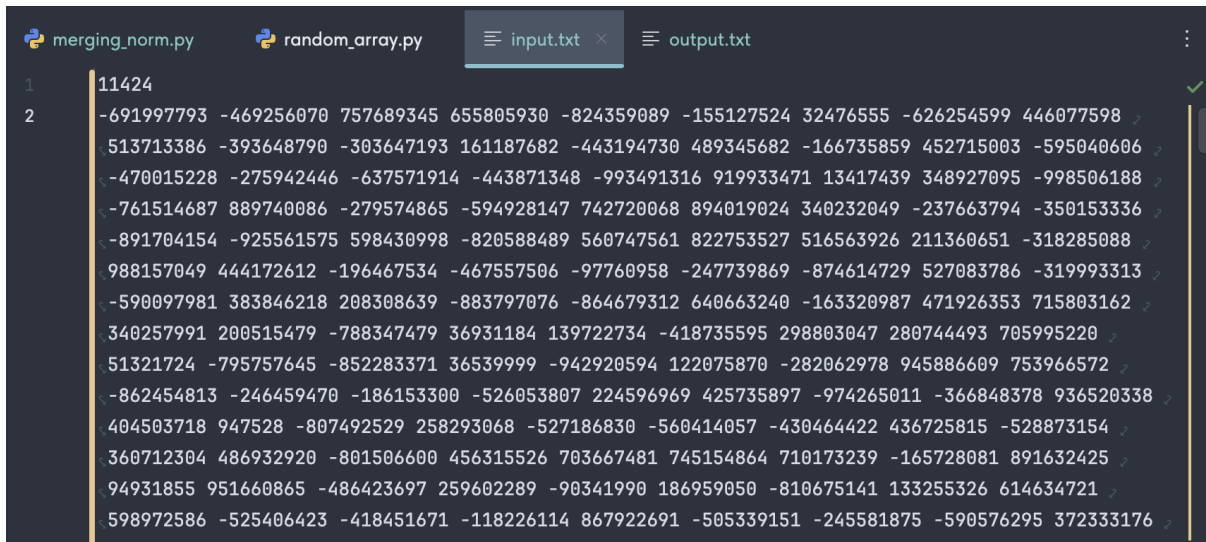
Рисунок 19 – Файл input.txt с входными данными худший случай
(убывание и элементы порядка 10^9 , 100000 элементов)

```

merging_norm.py random_array.py input.txt output.txt
1 1000036289 1000071499 1000117668 1000154899 1000249853 1000306872 1000345063 1000519072 1000526851
  1000746889 1000810554 1000813089 1001015933 1001054552 1001255377 1001472646 1001602615
  1001726706 1001758541 1001824756 1001824935 1002105965 1002219216 1002253739 1002283078
  1002301874 1002433229 1002559719 1002564273 1002851469 1003014976 1003101373 1003122215
  1003306569 1003430428 1003478732 1003763423 1003854400 1003881287 1004563796 1004582403
  1004784070 1004807711 1004881814 1005027595 1005075136 1005092588 1005127097 1005280318
  1005329767 1005333838 1005378707 1005382830 1005417684 1005463343 1005496954 1005667915
  1005768407 1005821706 1006148691 1006161483 1006219262 1006281935 1006525041 1006567623
  1006571068 1006598040 1006852534 1006934007 1006958251 1007338803 1007358485 1007386093
  1007740324 1007750792 1007957829 1008099070 1008149165 1008212132 1008376310 1008386105
  1008502789 1008525594 1008528177 1008567199 1008620649 1008647273 1008666692 1008703432
  1008707325 1008839271 1008927569 1008935678 1008952017 1009035831 1009050127 1009180958
  1009248974 1009295897 1009577708 1010011571 1010030405 1010058347 1010189639 1010289939
  1010429808 1010561149 1010589822 1010729900 1010785324 1010789130 1010863497 1011038151
  1011147735 1011359954 1011397748 1011493626 1011526930 1011667777 1011750143 1011980080

```

Рисунок 20 – Результат работы функции, худший случай (убывание и элементы порядка 10^9 , 100000 элементов)



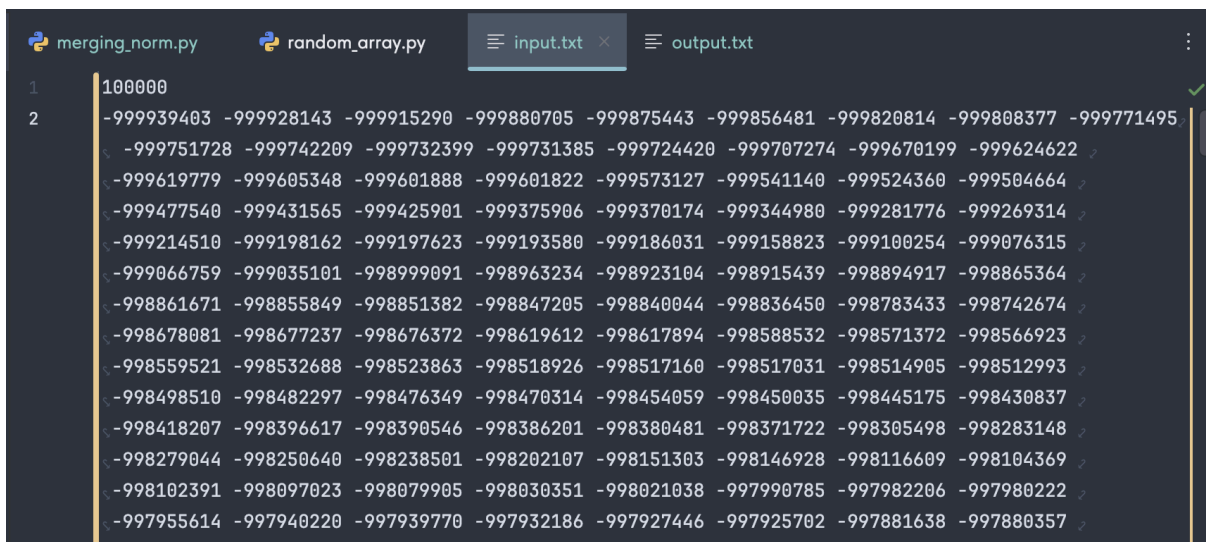
```
merging_norm.py random_array.py input.txt output.txt
1 11424
2 -691997793 -469256070 757689345 655805930 -824359089 -155127524 32476555 -626254599 446077598
513713386 -393648790 -303647193 161187682 -443194730 489345682 -166735859 452715003 -595040606
-470015228 -275942446 -637571914 -443871348 -993491316 919933471 13417439 348927095 -998506188
-761514687 889740086 -279574865 -594928147 742720068 894019024 340232049 -237663794 -350153336
-891704154 -925561575 598430998 -820588489 560747561 822753527 516563926 211360651 -318285088
988157049 444172612 -196467534 -467557506 -97760958 -247739869 -874614729 527083786 -319993313
-590097981 383846218 208308639 -883797076 -864679312 640663240 -163320987 471926353 715803162
340257991 200515479 -788347479 36931184 139722734 -418735595 298803047 280744493 705995220
51321724 -795757645 -852283371 36539999 -942920594 122075870 -282062978 945886609 753966572
-862454813 -246459470 -186153300 -526053807 224596969 425735897 -974265011 -366848378 936520338
404503718 947528 -807492529 258293068 -527186830 -560414057 -430464422 436725815 -528873154
360712304 486932920 -801506600 456315526 703667481 745154864 710173239 -165728081 891632425
94931855 951660865 -486423697 259602289 -90341990 186959050 -810675141 133255326 614634721
598972586 -525406423 -418451671 -118226114 867922691 -505339151 -245581875 -590576295 372333176
```

Рисунок 21 – Файл input.txt с входными данными средний случай



```
merging_norm.py random_array.py input.txt output.txt
1 -99991255 -999813820 -999807788 -999640199 -999410650 -999257007 -999231444 -999188834 -999137404
-999106071 -998917021 -998583813 -998506188 -998345051 -998053299 -997880436 -997858129
-997853468 -997818426 -997782105 -997745535 -997642030 -997468187 -997422023 -996960853
-996721701 -996700294 -996680920 -996306648 -996239503 -996087321 -996075555 -996000554
-995876947 -995853491 -995497851 -995295774 -995213539 -995177777 -994922890 -994501438
-994367054 -994287385 -993889037 -993795198 -993784963 -993695336 -993491316 -993139830
-992950182 -992949381 -992566582 -992141154 -992007815 -991666891 -991611568 -991564726
-991192461 -990972504 -990601970 -990014333 -989923468 -989909136 -989697987 -989588336
-989432792 -989285410 -989184539 -989072904 -988989735 -988686210 -988214986 -988172432
-988105474 -987891893 -987621030 -987561593 -987464183 -987413783 -987342859 -987166601
-987112618 -986991662 -986936959 -986786015 -986441091 -986397247 -986020270 -985784914
-985635918 -985531049 -985198389 -984640646 -984427247 -984365052 -984364905 -984304046
-984044507 -983952863 -983908292 -983893211 -983678812 -983346042 -983290735 -982465183
-982274790 -982222783 -982020691 -981934994 -981709929 -981350531 -981197524 -981057793
-981039880 -980877611 -980788644 -980724903 -980351952 -979969462 -979947829 -979662072
```

Рисунок 22 – Результат работы функции, средний случай



```
merging_norm.py random_array.py input.txt output.txt
1 100000
2 -999939403 -999928143 -999915290 -999880705 -999875443 -999856481 -999820814 -999808377 -999771495
-999751728 -999742209 -999732399 -999731385 -999724420 -999707274 -999670199 -999624622
-999619779 -999605348 -999601888 -999601822 -999573127 -999541140 -999524360 -999504664
-999477540 -999431565 -999425901 -999375906 -999370174 -999344980 -999281776 -999269314
-999214510 -999198162 -999197623 -999193580 -999186031 -999158823 -999100254 -999076315
-999066759 -999035101 -998999091 -998963234 -998923104 -998915439 -998894917 -998865364
-998861671 -998855849 -998851382 -998847205 -998840044 -998836450 -998783433 -998742674
-998678081 -998677237 -998676372 -998619612 -998617894 -998588532 -998571372 -998566923
-998559521 -998532688 -998523863 -998518926 -998517160 -998517031 -998514905 -998512993
-998498510 -998482297 -998476349 -998470314 -998454059 -998450035 -998445175 -998430837
-998418207 -998396617 -998390546 -998386201 -998380481 -998371722 -998305498 -998283148
-998279044 -998250640 -998238501 -998202107 -998151303 -998146928 -998116609 -998104369
-998102391 -998097023 -998079905 -998030351 -998021038 -997990785 -997982206 -997980222
-997955614 -997940220 -997939770 -997932186 -997927446 -997925702 -997881638 -997880357
```

Рисунок 23 – Файл input.txt с входными данными лучший случай

```

merging_norm.py random_array.py input.txt output.txt x
1 -999939403 -999928143 -999915290 -999880705 -999875443 -999856481 -999820814 -999808377 -999771495 ✓
  -999751728 -999742209 -999732399 -999731385 -999724420 -999707274 -999670199 -999624622
  -999619779 -999605348 -999601888 -999601822 -999573127 -999541140 -999524360 -999504664
  -999477540 -999431565 -999425901 -999375906 -999370174 -999344980 -999281776 -999269314
  -999214510 -999198162 -999197623 -999193580 -999186031 -999158823 -999100254 -999076315
  -999066759 -999035101 -998999091 -998963234 -998923104 -998915439 -998894917 -998865364
  -998861671 -998855849 -998851382 -998847205 -998840044 -998836450 -998783433 -998742674
  -998678081 -998677237 -998676372 -998619612 -998617894 -998588532 -998571372 -998566923
  -998559521 -998532688 -998523863 -998518926 -998517160 -998517031 -998514905 -998512993
  -998498510 -998482297 -998476349 -998470314 -998454059 -998450035 -998445175 -998430837
  -998418207 -998396617 -998390546 -998386201 -998380481 -998371722 -998305498 -998283148
  -998279044 -998250640 -998238501 -998202107 -998151303 -998146928 -998116609 -998104369
  -998102391 -998097023 -998079905 -998030351 -998021038 -997990785 -997982206 -997980222
  -997955614 -997940220 -997939770 -997932186 -997927446 -997925702 -997881638 -997880357
  -997869892 -997869571 -997849624 -997828358 -997824172 -997822310 -997803224 -997783465

```

Рисунок 24 – Результат работы функции, лучший случай

Результат работы кода на максимальных и минимальных значениях:

```

merging_norm.py random_array.py input.txt output.txt
1 1
2 937594352

```

Рисунок 25 – Файл input.txt с вводными данными – минимальными значениями

```

merging_norm.py random_array.py input.txt output.txt x
1 937594352

```

Рисунок 26 – Файл output.txt после выполнения функции с минимальными значениями

```

merging_norm.py random_array.py input.txt output.txt
1 20000 ✓
2 835146474 77623030 -401462753 909498849 -179722080 -599088405 981390287 359227637 341118937
  -378187791 -891961952 923519350 -938836962 409390939 -970114631 -388238326 783295301 -11467409
  872774703 -227730221 -647075828 714704436 436536136 369291081 670909588 929936433 997293531
  329605427 813396425 -793300597 239225779 -614517417 -267209471 -489626289 -138617259 80891197
  -481844304 -446591663 -241176065 -629211864 -44587092 56120402 730891513 -273187217 -59786360
  342499276 -682916948 353005955 -630451042 688847615 -698757653 -904147168 -972987146 -732014655
  754782451 -824340305 -659568408 287054730 984204921 -53985454 743396743 946969133 -765534109
  -239200430 254706427 -311891712 642008628 701255793 344449468 110771690 -977017588 -764588715
  -612907955 -136905520 -402528907 -89797879 -631953471 618845229 185467405 558158181 70049652
  824402024 -162977582 813350082 -11927906 -106986621 630900473 69075047 -985083779 -363401500
  696226950 246732259 928770334 79659999 -950356028 855760819 -396586676 -284767589 272932028
  -667136153 587741115 119859261 847547045 -551190276 38878509 -515917782 132302637 463203909
  -146464186 -534424394 605060858 -613910119 663951982 -465882831 9523994 -108111520 208054997
  -901398815 782724314 697149478 -86585051 663597605 950299636 -441063099 261793090 726101181

```

Рисунок 27 – Файл input.txt с вводными данными – максимальными значениями

```

merging_norm.py random_array.py input.txt output.txt x
1 -999875450 -999747557 -999585158 -999038072 -999007400 -998956502 -998860424 -998694262 -998607968 ✓
  -998544776 -998468709 -998380191 -998361151 -998326118 -998249525 -998105429 -998063959
  -998002598 -997957569 -997950602 -997854648 -997674471 -997604938 -997576928 -997560116
  -997526369 -997477874 -997262589 -997115753 -997058041 -996827386 -996604392 -996440670
  -996309920 -996164444 -996106887 -995969445 -995749903 -995733227 -995714098 -995618402
  -995486852 -995375041 -995213538 -995180630 -995177675 -994841677 -994814339 -994712362
  -994659210 -994599185 -994582656 -994568108 -994566309 -994535703 -994512041 -994279528
  -994202218 -994040263 -994016215 -993834712 -993734181 -993714488 -993641704 -993611674
  -993539040 -993512305 -993459912 -993456017 -993357486 -993073788 -992970124 -992933463
  -992755272 -992707029 -992485647 -992444300 -992385751 -992382065 -992315686 -992278603
  -992206339 -992088908 -992062349 -992058291 -991936808 -991863964 -991773409 -991729629
  -991639199 -991603146 -991574556 -991433613 -991130421 -990986486 -990527656 -990248361
  -990211091 -990050726 -990020063 -989891401 -989874372 -989702984 -989589210 -989538276
  -989528659 -989282404 -989192909 -989160303 -989086571 -988442928 -988301740 -988295463
  -988258758 -988231487 -988153175 -988001141 -987928550 -987810727 -987702491 -987553102
  
```

Рисунок 28 – Файл output.txt после выполнения функции с максимальными значениями

Таблица затрат памяти и времени по заданию 1 (измененный алгоритм):

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0004 сек	13.74 KB
Худший случай (убывание и элементы порядка 10^9 , 1000 элементов)	0.0014 сек	110.46 KB
Худший случай (убывание и элементы порядка 10^9 , 10000 элементов)	0.0103 сек	1085.85 KB
Худший случай (убывание и элементы порядка 10^9 , 100000 элементов)	0.1046сек	10746.88 KB
Средний случай	0.0119 сек	1179.40 KB
Лучший случай (отсортированный массив)	0.1016 сек	10236.92 KB
Верхняя граница диапазона значений входных данных из	0.0202 сек	2073.73 KB

Вывод по задаче: В этой задаче был реализован алгоритм сортировки слиянием. Сложность сортировки слиянием в худшем, среднем и лучшем случае составляет $O(n \cdot \log n)$, что делает ее предсказуемой и довольно надежной. За счет рекурсивной составляющей она требует больше памяти, чем сортировка вставками, но в рамки задания укладывается спокойно, а также отрабатывает на худшем случае со 100000 элементов довольно быстро, тогда как сортировка вставками не отрабатывает за приемлемое время (даже если выходить за рамки ограничений в задании).

Задача №2. Сортировка слиянием+

Дан массив целых чисел. Ваша задача — отсортировать его в порядке неубывания с помощью сортировки слиянием. Чтобы убедиться, что Вы действительно используете сортировку слиянием, мы просим Вас, после каждого осуществленного слияния (то есть, когда соответствующий подмассив уже отсортирован!), выводить индексы граничных элементов и их значения.

- Формат входного файла (input.txt). В первой строке входного файла содержится число n ($1 \leq n \leq 10^5$) — число элементов в массиве. Во второй строке находятся n различных целых чисел, по модулю не превосходящих 10^9 .

- Формат выходного файла (output.txt). Выходной файл состоит из нескольких строк.

- В последней строке выходного файла требуется вывести отсортированный в порядке неубывания массив, данный на входе. Между любыми двумя числами должен стоять ровно один пробел.

- Все предшествующие строки описывают осуществленные слияния, по одному на каждой строке. Каждая такая строка должна содержать по четыре числа: I_f , I_l , V_f , V_l , где I_f — индекс начала области слияния, I_l — индекс конца области слияния, V_f — значение первого элемента области слияния, V_l — значение последнего элемента области слияния.

- Все индексы начинаются с единицы (то есть, $1 \leq I_f \leq I_l \leq n$).

Индексы области слияния должны описывать положение области слияния в исходном массиве! Допускается не выводить информацию о слиянии для подмассива длиной 1, так как он отсортирован по определению.

- Ограничение по времени. 2сек.

- Ограничение по памяти. 256 мб.

- Приведем небольшой пример: отсортируем массив [9, 7, 5, 8]. Рекурсивная часть сортировки слиянием (процедура $\text{SORT}(A, L, R)$, где A — сортируемый массив, L — индекс начала области слияния, R — индекс конца области слияния) будет вызвана с $A = [9, 7, 5, 8]$, $L = 1$, $R = 4$ и выполнит следующие действия:

- разделит область слияния [1, 4] на две части, [1, 2] и [3, 4];

- выполнит вызов $\text{SORT}(A, L = 1, R = 2)$:

- * разделит область слияния [1, 2] на две части, [1, 1] и [2, 2];

- * получившиеся части имеют единичный размер, рекурсивные вызовы можно не делать;

- * осуществит слияние, после чего A станет равным [7, 9, 5, 8];
- * выведет описание слияния: $I_f = L = 1, I_l = R = 2, V_f = A_L = 7, V_l = A_R = 9$.
- выполнит вызов `SORT(A, L = 3, R = 4)`:
- * разделит область слияния [3, 4] на две части, [3, 3] и [4, 4];
- * получившиеся части имеют единичный размер, рекурсивные вызовы можно не делать;
- * осуществит слияние, после чего A станет равным [7, 9, 5, 8];
- * выведет описание слияния: $I_f = L = 3, I_l = R = 4, V_f = A_L = 5, V_l = A_R = 8$.
- осуществит слияние, после чего A станет равным [5, 7, 8, 9];
- выведет описание слияния: $I_f = L = 1, I_l = R = 4, V_f = A_L = 5, V_l = A_R = 9$.

input.txt	output.txt
10 1 8 2 1 4 7 3 2 3 6	1 2 1 8 3 4 1 2 1 4 1 8 5 6 4 7 1 6 1 8 7 8 2 3 9 10 3 6 7 10 2 6 1 10 1 8 1 1 2 2 3 3 4 6 7 8

– Описания слияний могут идти в произвольном порядке, необязательно совпадающем с порядком их выполнения. Однако, с целью повышения производительности, рекомендуем выводить эти описания сразу, не храня их в памяти. Именно по этой причине отсортированный массив выводится в самом конце.

• Пример:

Любая корректная сортировка слиянием, делящая подмассивы на две части (необязательно равных!), будет зачтена, если успеет завершиться, уложившись в ограничения.

Листинг кода.

```
from measurments import *
from random_array import *
from numpy import inf
```

```

@measure_performance
def get_data():
    with open('input.txt') as f:
        data = f.readlines()

    return int(data[0]), list(map(int, data[1].split()))

fin_data = []
def merge(arr, p, q, r):
    n1 = q - p
    n2 = r - q
    lft = [0 for _ in range(n1 + 1)]
    rght = [0 for _ in range(n2 + 1)]

    lft[:n1] = arr[p:q]
    rght[:n2] = arr[q:r]

    lft[-1] = inf
    rght[-1] = inf
    i, j = 0, 0

    for k in range(p, r):
        if lft[i] <= rght[j]:
            arr[k] = lft[i]
            i += 1
        else:
            arr[k] = rght[j]
            j += 1
    fin_data.append(f'{p + 1}, {r}, {arr[p]}, {arr[r-1]}\n')
    return arr

def merge_sort(arr, p, r):
    if p < r - 1:
        q = (p+r)//2
        merge_sort(arr, p, q)
        merge_sort(arr, q, r)
        return merge(arr, p, q, r)
    return arr

```

```

if __name__ == "__main__":
    generate_array(100000)
    ln, array = get_data()
    sarr = merge_sort(array, 0, ln)
    fin_data.append(" ".join(map(str, sarr)))
    with open('output.txt', 'w') as fl:
        fl.writelines(fin_data)
    assert sorted(array) == sarr, 'not sorted.....'

```

```

from measurments import *
from random_array import *
from numpy import inf

```

```

@measure_performance
def get_data():
    with open('input.txt') as f:
        data = f.readlines()

    return int(data[0]), list(map(int, data[1].split()))

```

```

fin_data = []
def merge(arr, p, q, r):
    n1 = q - p
    n2 = r - q
    lft = [0 for _ in range(n1 + 1)]
    rght = [0 for _ in range(n2 + 1)]

```

```

    lft[:n1] = arr[p:q]
    rght[:n2] = arr[q:r]

```

```

    lft[-1] = inf
    rght[-1] = inf
    i, j = 0, 0

```

```

    for k in range(p, r):
        if lft[i] <= rght[j]:
            arr[k] = lft[i]
            i += 1
        else:

```

```

        arr[k] = right[j]
        j += 1
    fin_data.append(f'{p + 1}, {r}, {arr[p]}, {arr[r-1]}\n')
    return arr

def merge_sort(arr, p, r):
    if p < r - 1:
        q = (p+r)//2
        merge_sort(arr, p, q)
        merge_sort(arr, q, r)
        return merge(arr, p, q, r)
    return arr

if __name__ == "__main__":
    generate_array(100000)
    ln, array = get_data()
    sarr = merge_sort(array, 0, ln)
    fin_data.append(" ".join(map(str, sarr)))
    with open('output.txt', 'w') as fl:
        fl.writelines(fin_data)
    assert sorted(array) == sarr, 'not sorted.....'

```

Текстовое объяснение решения.

Для выполнения второго задания были использованы `merge_sort()` и `merge()` из задания 1. Функция `merge_sort()` возвращает отсортированный массив чисел.

Для выполнения задания была введена переменная `fin_data` (список). В процессе слияния `fin_data` заполнялся строками с финальными данными: сортируемой частью массива, его первым и последним элементами после сортировки. По завершении выполнения сортировки список записывается в `output.txt`.

Результат работы кода на примерах из задания:

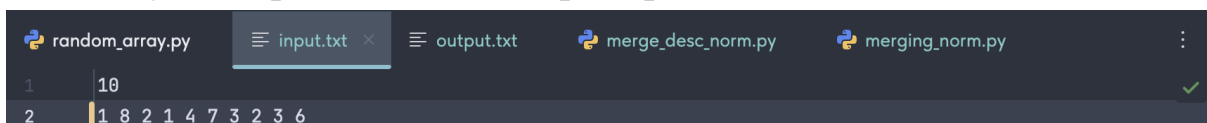


Рисунок 29 – Файл `input.txt` с входными данными из примера

```
random_array.py input.txt output.txt merge_desc_norm.py merging_norm.py
1 1, 2, 1, 8
2 4, 5, 1, 4
3 3, 5, 1, 4
4 1, 5, 1, 8
5 6, 7, 3, 7
6 9, 10, 3, 6
7 8, 10, 2, 6
8 6, 10, 2, 7
9 1, 10, 1, 8
10 1 1 2 2 3 3 4 6 7 8
```

Рисунок 30 – Результат работы кода для данных из примера, записанный в output.txt

```
random_array.py input.txt output.txt merge_desc_norm.py merging_norm.py
1 11826
2 360521244 120747469 -668150133 486754045 72751890 -6901930 -278758742 423386099 168067610
350720764 -684759504 -119420343 -588356555 -711324515 -288046136 -843550161 -89044432 88436677
548746774 346371419 688655994 389500556 -629949295 738794203 -154662060 -529485130 -911907432
642680726 45937792 -913513104 753088095 105230420 -396331360 -840049973 729095396 -468975278
-511816242 -519905414 834773085 -843112913 -888660673 850974543 641932166 509849841 937667246
699591813 -655107823 -417867040 918177128 805775631 -886161211 -346843963 160292463 -295949104
48220651 322341397 -276459190 220688184 -386282076 -32875812 749038485 -96505861 701254452
-872883479 -92456067 -390344108 -849546997 183234154 -98321314 437513520 -401591306 -568377089
-411567737 936248839 -10821114 351912393 -130457702 534405932 -440406236 -20921936 555431910
839494858 -170778461 -461188810 115329964 879824647 -817049005 -152970689 196520854 -582784603
188802385 -811148196 -720334929 -250881164 -750816720 885223037 -698896141 129247583 206521030
-440980340 -164710808 -683844734 -52930306 -939480913 -420369252 -34811309 -386472622 475517460
658923744 875777063 303931212 162770565 898332074 -795723478 -85203453 78571100 311208155
-86090562 667152529 -137249088 393337421 198300170 -418390894 -163707582 355191548 -280286460
```

Рисунок 31 – Файл input.txt со сгенерированными входными данными

```
random_array.py input.txt output.txt merge_desc_norm.py merging_norm.py
1 1, 2, 120747469, 360521244
2 4, 5, 72751890, 486754045
3 3, 5, -668150133, 486754045
4 1, 5, -668150133, 486754045
5 7, 8, -278758742, 423386099
6 6, 8, -278758742, 423386099
7 10, 11, -684759504, 350720764
8 9, 11, -684759504, 350720764
9 6, 11, -684759504, 423386099
10 1, 11, -684759504, 486754045
11 13, 14, -711324515, -588356555
12 12, 14, -711324515, -119420343
13 16, 17, -843550161, -89044432
14 15, 17, -843550161, -89044432
15 12, 17, -843550161, -89044432
```

Рисунок 32 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:

```

1 1
2 -1223344

```

Рисунок 33 – Минимальное вводимое значение в input.txt

```

1 1 -1223344

```

Рисунок 34 – Результат работы кода для значения $n = 1$, записанный в output.txt

```

1 100000
2 316378742 -412181730 -30074885 -16467878 53223595 588383807 -767614797 422407955 941677956
3 99274228 732757963 778255579 -391742902 205755413 -363446943 290559077 973482410 -470318958
4 258064073 -655556460 -98822742 -257831194 639182471 751158938 138369045 186634125 907003180
5 -512950520 260014330 672180244 -150103820 -483842131 111224245 -726451741 -946636480 616456425
6 -370876311 225253068 152371302 559102786 -909982396 115652260 298751015 201771483 373878772
7 873254891 -956182201 656873562 87854585 129737021 483558260 682031655 -835550588 369668430
8 -42165530 735940713 -94606284 626017748 860619114 -510007985 478783339 -893805523 -510590527
9 237643524 619489321 359209467 756211995 429656446 -417059901 399407970 43734498 -8653924 19414862
10 -57959603 -547928978 -446011083 853862546 214948885 -944181925 942682335 524394256 813110464
11 564110819 -920069458 772248547 878498881 540558700 -70321198 -318407335 990965526 128582482
12 -625037087 -464524180 637894853 789707416 715344039 -675501893 350988280 -370360798 -39887722
13 556647695 -205641709 456615606 602698152 909618038 -769531212 -29219196 10525606 -588006001
14 -811030434 66201724 714483297 -686201517 89540742 970822950 -154843755 323568850 187862432
15 226274839 -659922977 121823324 -315654161 867838651 955554032 -213480952 199378706 -935121847

```

Рисунок 35 – Максимальное вводимое значение в input.txt

```

1 2, 3, -412181730, -30074885
2 1, 3, -412181730, -30074885
3 5, 6, 53223595, 588383807
4 4, 6, -16467878, 588383807
5 1, 6, -412181730, 588383807
6 8, 9, 422407955, 941677956
7 7, 9, -767614797, 941677956
8 11, 12, 732757963, 778255579
9 10, 12, 732757963, 789274228
10 7, 12, -767614797, 941677956
11 1, 12, -767614797, 941677956
12 14, 15, -363446943, 205755413
13 13, 15, -391742902, 205755413

```

Рисунок 36 – Результат работы кода для значения $n = 1000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 2:

	Время выполнения	Затраты памяти
--	------------------	----------------

Нижняя граница диапазона значений входных данных из текста задачи	0.0004 сек	13.74 KB
Пример из задачи	0.0005 сек	13.69 KB
Собственный пример	0.0159 сек	1214.02 KB
Верхняя граница диапазона значений входных данных из текста задачи	0.1089 сек	10236.55 KB

Вывод по задаче: В задании к функции сортировки слиянием был добавлен алгоритм фиксации части слияния, ее индексов (начало и конец) в изначальном массиве.

Задача №9. Метод Штрассена для умножения матриц

Умножение матриц. Простой метод. Если есть квадратные матрицы

$X = (x_{ij})$ и $Y = (y_{ij})$, то их произведение $Z = X \cdot Y \Rightarrow z_{ij} = \sum_{k=1}^n x_{ik} \cdot y_{kj}$. Нужно

вычислить n^2 элементов матрицы, каждый из которых представляет собой сумму n значений.

Matrix_Multiply(X, Y):

$n = X.rows$

Z - квадратная матрица размера n

for $i = 1$ to n :

for $j = 1$ to n :

$z[i,j] = 0$

for $k = 1$ to n :

$z[i,j] = z[i,j] + x[i,k] * y[k,j]$

return Z

Задачу умножения матриц достаточно легко разбить на подзадачи, поскольку произведение можно составлять из блоков. Разобьём каждую из матриц X и Y на четыре блока размера $\frac{n}{2} \times \frac{n}{2}$

$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix}, Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix},$$

Тогда их произведение выражается в терминах этих блоков по обычной формуле умножения матриц :

$$XY = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \cdot \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}$$

Вычислив рекурсивно восемь произведений $AE, BG, AF, BH, CE, DG, CF, DH$ и просуммировав их за время $O(n^2)$, мы вычислим необходимое нам произведение матриц. Соответствующее рекуррентное соотношение на время работы алгоритма

$$T(n) = 8T(n/2) + O(n^2).$$

Какое получилось время у предыдущего рекурсивного алгоритма? Да, ничуть не лучше наивного. Однако его можно ускорить с помощью алгебраического трюка: для вычисления произведения XY достаточно перемножить семь пар матриц размера $\frac{n}{2} \times \frac{n}{2}$, после чего хитрым образом (и как только Штрассен догадался?)

получить ответ:

$$XY = \begin{bmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{bmatrix}$$

где

$P_1 = A(F-H)$, $P_5 = (A+D)(E+H)$,

$P_2 = (A+B)H$, $P_6 = (B-D)(G+H)$,

$P_3 = (C+D)E$, $P_7 = (A-C)(E+F)$,

$P_4 = D(G-E)$.

- Цель. Применить метод Штрассена для умножения матриц и сравнить его с простым методом. Найти размер матриц n , при котором метод Штрассена работает существенно быстрее простого метода.

- Формат входа. Стандартный ввод или input.txt. Первая строка - размер квадратных матриц n для умножения. Следующие строки соответственно сами значения матриц A и B .

- Формат выхода. Стандартный вывод или output.txt. Матрица $C=A \cdot B$.

Листинг кода.

```
from random_matrix import *
import numpy as np
from measurments import *

def get_data():
    with open('input.txt') as f:
        data = f.readlines()
        n, pad = int(data[0]), int(data[1])
        mat_a, mat_b = np.array([list(map(int, i.split())) for i in data[2:pad+2]]),
np.array([list(map(int, i.split())) for i in data[pad+2:]])
        return n, pad, mat_a, mat_b

@measure_performance
def shtrassen_multiplication(n, x, y, rn=None):
    if n >= 2:
        q = n // 2

        a = np.array([i[:q] for i in x[:q]])
        b = np.array([i[q:] for i in x[:q]])
        c = np.array([i[:q] for i in x[q:]])
```

```

d = np.array([i[q:] for i in x[q:]])

e = np.array([i[:q] for i in y[:q]])
f = np.array([i[q:] for i in y[q:]])
g = np.array([i[:q] for i in y[q:]])
h = np.array([i[q:] for i in y[q:]])

p1 = shtrassen_multiplication(len(a), a, matrix_sum(f, -1 * h))
p2 = shtrassen_multiplication(len(h), matrix_sum(a, b), h)
p3 = shtrassen_multiplication(len(e), matrix_sum(c, d), e)
p4 = shtrassen_multiplication(len(d), d, matrix_sum(g, (-1)*e))
p5 = shtrassen_multiplication(len(a), matrix_sum(a, d),
matrix_sum(e, h))
p6 = shtrassen_multiplication(len(b), matrix_sum(b, -1 * d),
matrix_sum(g, h))
p7 = shtrassen_multiplication(len(a), matrix_sum(a, -1 * c),
matrix_sum(e, f))

width = len(p5[0]) + len(p1[0])
height = len(p5) + len(p3)

res = [[0 for _ in range(width)] for _ in range(height)]
l = matrix_sum(p5, matrix_sum(p4, matrix_sum(-1 * p2, p6)))
m = matrix_sum(p1, p2)
p = matrix_sum(p3, p4)
o = matrix_sum(p1, matrix_sum(p5, matrix_sum(-1 * p3, -1 * p7)))

for i in range(height // 2):
    for j in range(width // 2):
        res[i][j] = l[i][j]

for i in range(height // 2):
    for j in range(width // 2, width):
        res[i][j] = m[i][j - width // 2]

for i in range(height // 2, height):
    for j in range(width // 2):
        res[i][j] = p[i - height // 2][j]

```

```

        for i in range(height // 2, height):
            for j in range(width // 2, width):
                res[i][j] = o[i - height // 2][j - width // 2]
            return np.array([i[:rn] for i in res[:rn]]) if rn else np.array(res)

    return np.array([x[0] * y[0]])

def matrix_sum(a, b):
    n = len(a)
    c = np.array([[0 for _ in range(n)] for _ in range(n)])
    for i in range(n):
        for j in range(n):
            c[i][j] = a[i][j] + b[i][j]
    return c

@measure_performance
def base_matmul(n, x, y, rn=None):
    if n >= 2:
        q = n // 2

        a = np.array([i[:q] for i in x[:q]])
        b = np.array([i[q:] for i in x[:q]])
        c = np.array([i[:q] for i in x[q:]])
        d = np.array([i[q:] for i in x[q:]])

        e = np.array([i[:q] for i in y[:q]])
        f = np.array([i[q:] for i in y[:q]])
        g = np.array([i[:q] for i in y[q:]])
        h = np.array([i[q:] for i in y[q:]])

        ae = base_matmul(len(a), a, e)
        bg = base_matmul(len(b), b, g)
        af = base_matmul(len(a), a, f)
        bh = base_matmul(len(b), b, h)
        ce = base_matmul(len(c), c, e)
        dg = base_matmul(len(d), d, g)
        cf = base_matmul(len(c), c, f)
        dh = base_matmul(len(d), d, h)

```

```

width = len(ae[0]) + len(af[0])
height = len(ae) + len(ce)

res = [[0 for _ in range(width)] for _ in range(height)]
l = matrix_sum(ae, bg)
m = matrix_sum(af, bh)
p = matrix_sum(ce, dg)
o = matrix_sum(cf, dh)

for i in range(height // 2):
    for j in range(width // 2):
        res[i][j] = l[i][j]

for i in range(height // 2):
    for j in range(width // 2, width):
        res[i][j] = m[i][j - width // 2]

for i in range(height // 2, height):
    for j in range(width // 2):
        res[i][j] = p[i - height // 2][j]

for i in range(height // 2, height):
    for j in range(width // 2, width):
        res[i][j] = o[i - height // 2][j - width // 2]
return np.array([i[:rn] for i in res[:rn]]) if rn else np.array(res)

return np.array([x[0] * y[0]])

if __name__ == "__main__":
    for h in range(1, 100):
        generate_data(h)
        ln, p, mat1, mat2 = get_data()
        fin = shtrassen_multiplication(p, mat1, mat2, ln)
        base_fin = base_matmul(p, mat1, mat2, ln)
        mat1 = np.array([i[:ln] for i in mat1[:ln]])
        mat2 = np.array([i[:ln] for i in mat2[:ln]])
        real_res = mat1 @ mat2
        with open('output.txt', 'w') as fl:

```

```

        fl.writelines("\n".join(map(str, [" ".join(map(str, row)) for row in
fin])))

        assert all([real_res[i][j] == fin[i][j] == base_fin[i][j] for j in
range(ln)] for i in range(ln)), 'not equal.....'
        print('--' * 50)

```

```

from random_matrix import *
import numpy as np
from measurments import *

def get_data():
    with open('input.txt') as f:
        data = f.readlines()
    n, pad = int(data[0]), int(data[1])
    mat_a, mat_b = np.array([list(map(int, i.split())) for i in data[2:pad+2]]),
np.array([list(map(int, i.split())) for i in data[pad+2:]])
    return n, pad, mat_a, mat_b

@measure_performance
def shtrassen_multiplication(n, x, y, rn=None):
    if n >= 2:
        q = n // 2

        a = np.array([i[:q] for i in x[:q]])
        b = np.array([i[q:] for i in x[:q]])
        c = np.array([i[:q] for i in x[q:]])
        d = np.array([i[q:] for i in x[q:]])

        e = np.array([i[:q] for i in y[:q]])
        f = np.array([i[q:] for i in y[:q]])
        g = np.array([i[:q] for i in y[q:]])
        h = np.array([i[q:] for i in y[q:]])

        p1 = shtrassen_multiplication(len(a), a, matrix_sum(f, -1 * h))
        p2 = shtrassen_multiplication(len(h), matrix_sum(a, b), h)
        p3 = shtrassen_multiplication(len(e), matrix_sum(c, d), e)
        p4 = shtrassen_multiplication(len(d), d, matrix_sum(g, (-1)*e))
        p5 = shtrassen_multiplication(len(a), matrix_sum(a, d),
matrix_sum(e, h))

```

```

        p6 = shtrassen_multiplication(len(b), matrix_sum(b, -1 * d),
matrix_sum(g, h))
        p7 = shtrassen_multiplication(len(a), matrix_sum(a, -1 * c),
matrix_sum(e, f))

```

```

width = len(p5[0]) + len(p1[0])
height = len(p5) + len(p3)

```

```

res = [[0 for _ in range(width)] for _ in range(height)]
l = matrix_sum(p5, matrix_sum(p4, matrix_sum(-1 * p2, p6)))
m = matrix_sum(p1, p2)
p = matrix_sum(p3, p4)
o = matrix_sum(p1, matrix_sum(p5, matrix_sum(-1 * p3, -1 * p7)))

```

```

for i in range(height // 2):
    for j in range(width // 2):
        res[i][j] = l[i][j]

```

```

for i in range(height // 2):
    for j in range(width // 2, width):
        res[i][j] = m[i][j - width // 2]

```

```

for i in range(height // 2, height):
    for j in range(width // 2):
        res[i][j] = p[i - height // 2][j]

```

```

for i in range(height // 2, height):
    for j in range(width // 2, width):
        res[i][j] = o[i - height // 2][j - width // 2]
return np.array([i[:rn] for i in res[:rn]]) if rn else np.array(res)

```

```

return np.array([x[0] * y[0]])

```

```

def matrix_sum(a, b):
    n = len(a)
    c = np.array([[0 for _ in range(n)] for _ in range(n)])
    for i in range(n):
        for j in range(n):
            c[i][j] = a[i][j] + b[i][j]

```

```
return c
```

```
@measure_performance
```

```
def base_matmul(n, x, y, rn=None):
```

```
    if n >= 2:
```

```
        q = n // 2
```

```
        a = np.array([i[:q] for i in x[:q]])
```

```
        b = np.array([i[q:] for i in x[:q]])
```

```
        c = np.array([i[:q] for i in x[q:]])
```

```
        d = np.array([i[q:] for i in x[q:]])
```

```
        e = np.array([i[:q] for i in y[:q]])
```

```
        f = np.array([i[q:] for i in y[:q]])
```

```
        g = np.array([i[:q] for i in y[q:]])
```

```
        h = np.array([i[q:] for i in y[q:]])
```

```
        ae = base_matmul(len(a), a, e)
```

```
        bg = base_matmul(len(b), b, g)
```

```
        af = base_matmul(len(a), a, f)
```

```
        bh = base_matmul(len(b), b, h)
```

```
        ce = base_matmul(len(c), c, e)
```

```
        dg = base_matmul(len(d), d, g)
```

```
        cf = base_matmul(len(c), c, f)
```

```
        dh = base_matmul(len(d), d, h)
```

```
        width = len(ae[0]) + len(af[0])
```

```
        height = len(ae) + len(ce)
```

```
        res = [[0 for _ in range(width)] for _ in range(height)]
```

```
        l = matrix_sum(ae, bg)
```

```
        m = matrix_sum(af, bh)
```

```
        p = matrix_sum(ce, dg)
```

```
        o = matrix_sum(cf, dh)
```

```
        for i in range(height // 2):
```

```
            for j in range(width // 2):
```

```
                res[i][j] = l[i][j]
```

```

    for i in range(height // 2):
        for j in range(width // 2, width):
            res[i][j] = m[i][j - width // 2]

    for i in range(height // 2, height):
        for j in range(width // 2):
            res[i][j] = p[i - height // 2][j]

    for i in range(height // 2, height):
        for j in range(width // 2, width):
            res[i][j] = o[i - height // 2][j - width // 2]
    return np.array([i[:rn] for i in res[:rn]]) if rn else np.array(res)

return np.array([x[0] * y[0]])

if __name__ == "__main__":
    for h in range(1, 100):
        generate_data(h)
        ln, p, mat1, mat2 = get_data()
        fin = shtrassen_multiplication(p, mat1, mat2, ln)
        base_fin = base_matmul(p, mat1, mat2, ln)
        mat1 = np.array([i[:ln] for i in mat1[:ln]])
        mat2 = np.array([i[:ln] for i in mat2[:ln]])
        real_res = mat1 @ mat2
        with open('output.txt', 'w') as fl:
            fl.writelines("\n".join(map(str, [" ".join(map(str, row)) for row in
fin])))

        assert all([real_res[i][j] == fin[i][j] == base_fin[i][j] for j in
range(ln)] for i in range(ln)), 'not equal.....'
        print('--' * 50)

```

Листинг кода дополнительных функций

```

import random
import numpy as np
import math

def generate_data(n=None):
    if not n:
        n = random.randint(1, 3)

```

```

if not np.log2(n).is_integer():
    k = math.ceil(np.log2(n))
    p = 2 ** k
    a = [[random.randint(-10**4, 10**4) for _ in range(n)] for _ in
range(n)]
    b = [[random.randint(-10**4, 10**4) for _ in range(n)] for _ in
range(n)]
    padw = (p-n)
    a_fin = np.pad(a, ((0, padw), (0, padw)), 'constant')
    b_fin = np.pad(b, ((0, padw), (0, padw)), 'constant')
    matrix_a = [f'{" ".join(map(str, row))}\n' for row in a_fin]
    matrix_b = [f'{" ".join(map(str, row))}\n' for row in b_fin]
else:
    matrix_a = [f'{" ".join(map(str, [random.randint(-10**4, 10**4) for _
in range(n)]))}\n' for _ in range(n)]
    matrix_b = [f'{" ".join(map(str, [random.randint(-10**4, 10**4) for _
in range(n)]))}\n' for _ in range(n)]
    p = n
    all_data = [f'{n}\n', f'{p}\n', f'{" ".join(map(str, matrix_a))}', f'{"
".join(map(str, matrix_b))}']

    with open('input.txt', 'w') as f:
        f.writelines(all_data)

```

Текстовое объяснение решения.

Для решения задания было написано несколько функций: `get_data()` для извлечения информации из файла, `shtrassen_multiplication()` и `matrix_sum()` для выполнения умножения по методу Штрассена, `generate_data()` для генерации данных для тестирования функции.

Функция `generate_data()` из аргументов принимает только размерность матрицы. Если размерность не передана, то генерируется любое число от 1 до 25 (можно выставить любое ограничение). Если полученное `n` не является степенью двойки, что важно для метода Штрассена, то ищется ближайшая степень двойки, которая больше числа, и сгенерированная матрица с правого и нижнего краев дополняется нулями до необходимого размера. Сгенерированные матрицы записываются в файл, перед ними записываются оригинальный и дополненный размер матриц.

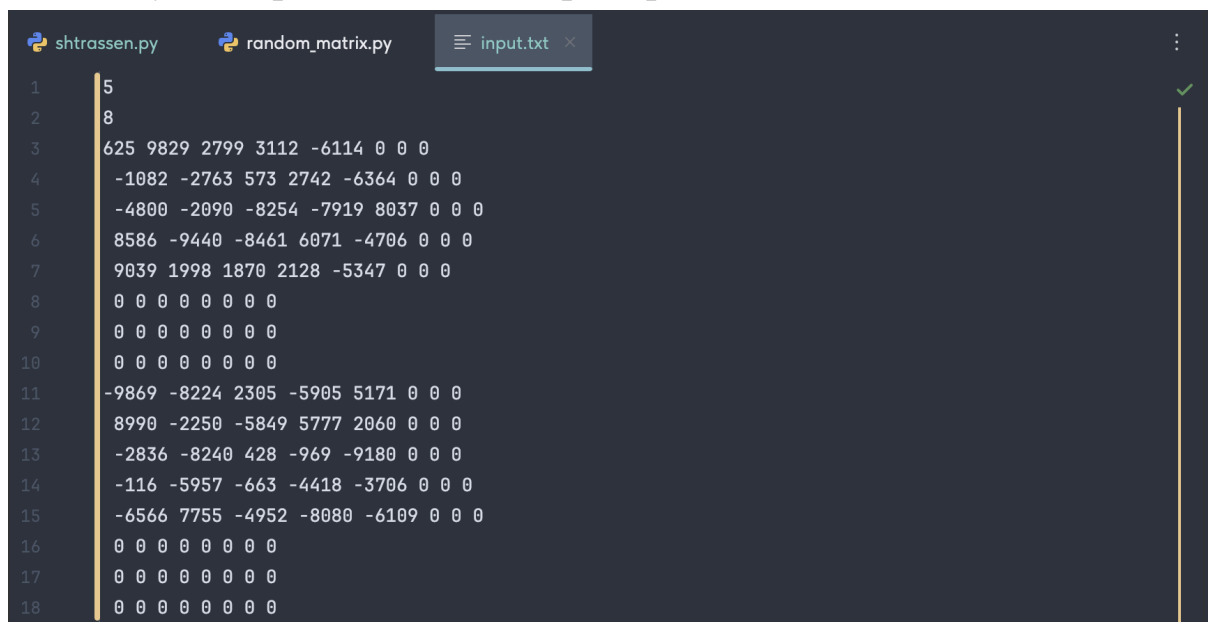
Функция `shtrassen_multiplication()` принимает следующие аргументы: переданный размер матриц, матрица А, матрица В и их реальный размер. Далее каждая из матриц делится на левый верхний, правый верхний, левый нижний и правый нижний блоки, полученные матрицы отправляются на процедуру деления рекурсивно. По достижении размерности 1 возвращается произведение элементов матриц. Потом матрицы рекурсивно считаются по данным в задании формулам (при каждом разбиении, когда размерность больше 1). Далее вычисляются длина и высота результирующей матрицы, создается сама результирующая матрица из нулей и заполняется по блокам, на которые была разделена.

Если реальный размер не совпадает с переданным, то результирующая матрица обрезается до истинного размера и возвращается.

Функция `matrix_sum()` поэлементно складывает матрицы с помощью цикла `for`. Возвращает результат сложения.

После выполнения функции проводится проверка с помощью встроенной в Python функции матричного умножения (`@`), а также (ради интереса и по заданию) производится сравнение с функцией `base_matmul()` по производительности.

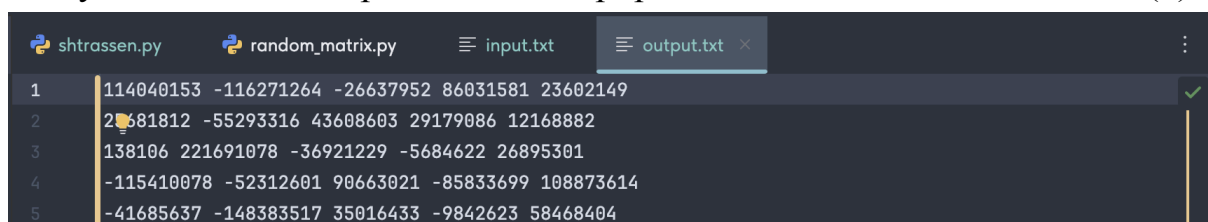
Результат работы кода на примерах из задания:



```

1 5
2 8
3 625 9829 2799 3112 -6114 0 0 0
4 -1082 -2763 573 2742 -6364 0 0 0
5 -4800 -2090 -8254 -7919 8037 0 0 0
6 8586 -9440 -8461 6071 -4706 0 0 0
7 9039 1998 1870 2128 -5347 0 0 0
8 0 0 0 0 0 0 0 0
9 0 0 0 0 0 0 0 0
10 0 0 0 0 0 0 0 0
11 -9869 -8224 2305 -5905 5171 0 0 0
12 8990 -2250 -5849 5777 2060 0 0 0
13 -2836 -8240 428 -969 -9180 0 0 0
14 -116 -5957 -663 -4418 -3706 0 0 0
15 -6566 7755 -4952 -8080 -6109 0 0 0
16 0 0 0 0 0 0 0 0
17 0 0 0 0 0 0 0 0
18 0 0 0 0 0 0 0 0
  
```

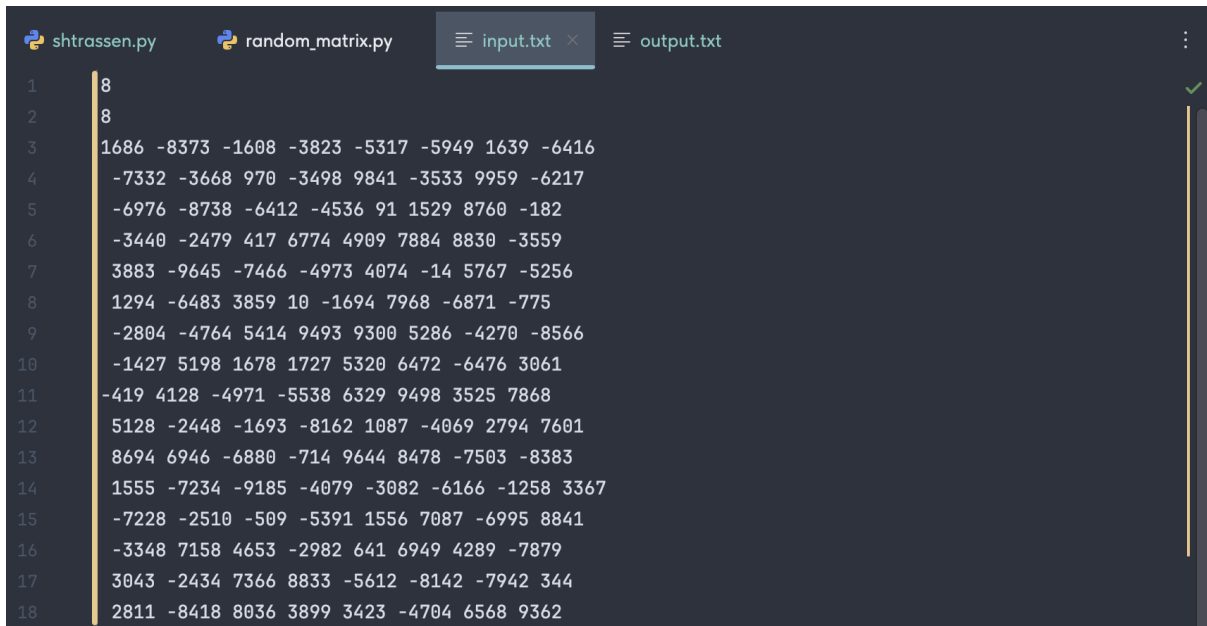
Рисунок 37 – Файл `input.txt` со сгенерированными входными данными (1)



```

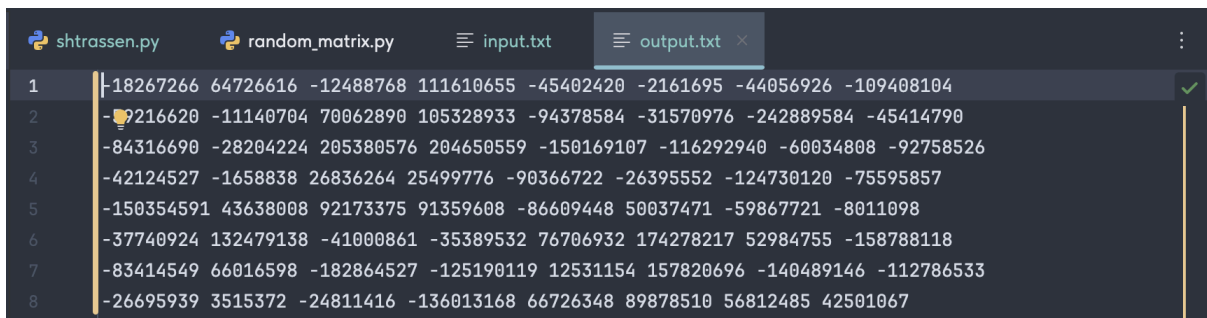
1 114040153 -116271264 -26637952 86031581 23602149
2 25681812 -55293316 43608603 29179086 12168882
3 138106 221691078 -36921229 -5684622 26895301
4 -115410078 -52312601 90663021 -85833699 108873614
5 -41685637 -148383517 35016433 -9842623 58468404
  
```

Рисунок 38 – Результат работы кода для сгенерированных данных (1), записанный в output.txt



```
shtrassen.py random_matrix.py input.txt output.txt
1 8
2 8
3 1686 -8373 -1608 -3823 -5317 -5949 1639 -6416
4 -7332 -3668 970 -3498 9841 -3533 9959 -6217
5 -6976 -8738 -6412 -4536 91 1529 8760 -182
6 -3440 -2479 417 6774 4909 7884 8830 -3559
7 3883 -9645 -7466 -4973 4074 -14 5767 -5256
8 1294 -6483 3859 10 -1694 7968 -6871 -775
9 -2804 -4764 5414 9493 9300 5286 -4270 -8566
10 -1427 5198 1678 1727 5320 6472 -6476 3061
11 -419 4128 -4971 -5538 6329 9498 3525 7868
12 5128 -2448 -1693 -8162 1087 -4069 2794 7601
13 8694 6946 -6880 -714 9644 8478 -7503 -8383
14 1555 -7234 -9185 -4079 -3082 -6166 -1258 3367
15 -7228 -2510 -509 -5391 1556 7087 -6995 8841
16 -3348 7158 4653 -2982 641 6949 4289 -7879
17 3043 -2434 7366 8833 -5612 -8142 -7942 344
18 2811 -8418 8036 3899 3423 -4704 6568 9362
```

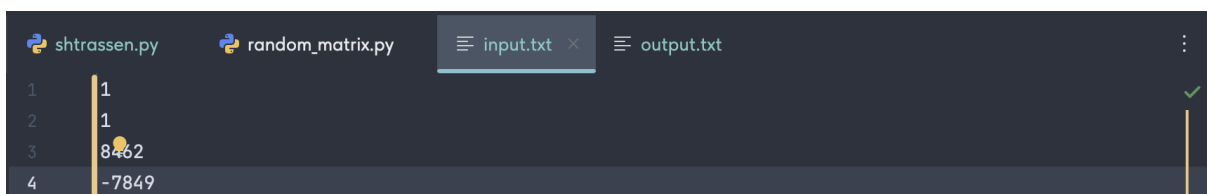
Рисунок 39 – Файл input.txt со сгенерированными входными данными (2)



```
shtrassen.py random_matrix.py input.txt output.txt
1 -18267266 64726616 -12488768 111610655 -45402420 -2161695 -44056926 -109408104
2 -9216620 -11140704 70062890 105328933 -94378584 -31570976 -242889584 -45414790
3 -84316690 -28204224 205380576 204650559 -150169107 -116292940 -60034808 -92758526
4 -42124527 -1658838 26836264 25499776 -90366722 -26395552 -124730120 -75595857
5 -150354591 43638008 92173375 91359608 -86609448 50037471 -59867721 -8011098
6 -37740924 132479138 -41000861 -35389532 76706932 174278217 52984755 -158788118
7 -83414549 66016598 -182864527 -125190119 12531154 157820696 -140489146 -112786533
8 -26695939 3515372 -24811416 -136013168 66726348 89878510 56812485 42501067
```

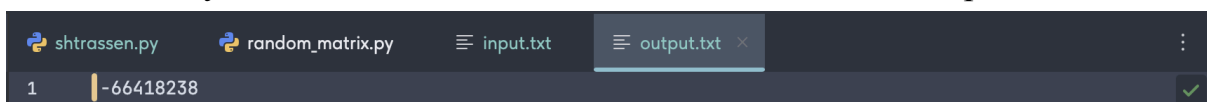
Рисунок 40 – Результат работы кода для сгенерированных данных (2), записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:



```
shtrassen.py random_matrix.py input.txt output.txt
1 1
2 1
3 8462
4 -7849
```

Рисунок 41 – Минимальное вводимое значение в input.txt



```
shtrassen.py random_matrix.py input.txt output.txt
1 -66418238
```

Рисунок 42 – Результат работы кода для значения $n = 1$, записанный в output.txt

```

shtrassen.py random_matrix.py input.txt output.txt
1 100
2 128
3 1880 3604 1587 2030 5448 7033 7146 -6227 4361 -1979 2237 3846 -4932 1035 8423 3556 7855 6833 5508 -6861 6
4 -4482 8172 1107 -9467 -3061 2752 7913 -6517 2295 -1213 -72 3617 -7418 4501 4507 3794 -6746 -2399 -527 47
5 -9638 1378 7648 -1814 7266 -560 8397 4269 -6995 -1467 -7501 -5833 149 1246 -7446 -6071 440 -2 -2794 3406
6 -6240 -4963 6133 -6200 -7998 -2862 2413 -1773 1150 -709 -9841 -4277 8303 3393 -3618 5016 -7960 8196 -324
7 -453 -9863 6530 -4575 1967 4972 -9621 1698 9375 -4648 -4349 -9660 -6286 -3300 -6082 4314 6813 8551 -2423
8 -7210 1185 -6973 -9875 -9980 -17 8057 4614 9843 9890 -3867 -325 -1161 -6695 -9054 5608 -7404 1812 1131 4
9 9780 6746 -7614 -9373 3367 4391 -277 -8807 5133 996 2939 -5935 -8414 -9720 3088 7206 -103 6945 -1877 505
10 -2573 -4987 7262 5605 6519 -3327 -7217 8406 -642 -7149 -1061 -7186 3510 1028 7725 4804 -2787 -3266 3486

```

Рисунок 43 – Максимальное вводимое значение в input.txt

```

shtrassen.py random_matrix.py input.txt output.txt
1 279262587 -43402937 -99890703 482380253 -576918833 681393860 131605058 -541514982 -373646584 76112413 ✓
2 -35075826 582853017 481419563 -235817865 604449783 -115684999 -55237151 60984229 375713250 261864828 785
3 318399491 -139537798 -20223829 43106888 66853693 179898452 144798538 -68765610 -248791488 -102389765 -594
4 248506411 -158599207 338584353 -479873314 524445703 -311547926 279115468 237088741 -321362506 -89597869 -
5 -83628828 179085360 -133387037 -161260224 -66667573 -215784277 -48336281 33532517 649057923 -317042378 36
6 -283644278 20160533 173810771 -14817849 355878526 -22908946 -318806121 20572197 -504495774 899442426 6231
7 -450890435 666736902 137088637 535773456 166649195 740207971 -181551295 -433509238 81955157 14089592 -186
8 483734690 -273669205 -174941122 -653033390 -639367277 127395756 31486591 -475869601 26979289 -517323496 2
9 -296166130 91360181 -651036701 161447554 -375887372 291470556 716639421 -621405521 -204457329 -268141423
10 377749801 355749655 -235047817 350362457 106621533 -296502305 235351855 420864093 244433969 -408123507 -2

```

Рисунок 44 – Результат работы кода для значения $n = 1000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 9:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0002 сек	0.30 KB
Собственный пример 1	0.0408 сек	9.89 KB
Собственный пример 2	0.0792 сек	9.95 KB
Верхняя граница диапазона значений входных данных из текста задачи	112.1723 сек	1297.24 KB

Таблица сравнения затрат времени и памяти для умножения методом Штрассена и базовым методом

	Метод Штрассена		Базовый метод	
	Время выполнения	Затраты памяти	Время выполнения	Затраты памяти

	ия		я	
Матрица размерности 2	0.0009 сек	3.27 KB	0.0005 сек	2.97 KB
Матрица размерности 4	0.0057 сек	5.55 KB	0.0038 сек	5.25 KB
Матрица размерности 8	0.0421 сек	9.52 KB	0.0325 сек	9.00 KB
Матрица размерности 16	0.7272 сек	24.83 KB	0.3296 сек	25.33 KB
Матрица размерности 32	2.1664 сек	90.08 KB	2.0691 сек	92.08 KB
Матрица размерности 64	15.8479 сек	352.37 KB	20.7887 сек	360.37 KB
Матрица размерности 128	121.1561 сек	1398.74 KB	150.5585 сек	1430.74 KB
Матрица размерности 256	821.9128 сек	5570.93 KB	1133.4211 сек	5698.93 KB

Вывод по задаче: Умножение матриц по методу Штрассена становится эффективным, когда размерность матриц превышает значение 32. С этого момента время выполнения методом Штрассена всегда меньше примерно 5-40 сек, а потребление памяти – на 20-40 килобайт.

Задачи на продвинутый уровень

Задача №4. Бинарный поиск

В этой задаче вы реализуете алгоритм бинарного поиска, который позволяет очень эффективно искать (даже в огромных) списках при условии, что список отсортирован. Цель - реализация алгоритма двоичного (бинарного) поиска.

- Формат входного файла (input.txt). В первой строке входного файла содержится число n ($1 \leq n \leq 10^5$) — число элементов в массиве, и последовательность $a_0 < a_1 < \dots < a_{n-1}$ из n различных положительных целых чисел в порядке возрастания, $1 \leq a_i \leq 10^9$ для всех $0 \leq i < n$. Следующая строка содержит число k , $1 \leq k \leq 10^5$ и k положительных целых чисел $b_0, \dots, b_{k-1}$, $1 \leq b_j \leq 10^9$ для всех $0 \leq j < k$.

- Формат выходного файла (output.txt). Для всех i от 0 до $k-1$ вывести индекс $0 \leq j \leq n-1$, такой что $a_i = b_j$ или -1, если такого числа в массиве нет.

- Ограничение по времени. 2сек.

- Ограничение по памяти. 256 мб.

- Пример:

input.txt	output.txt
5	2 0 -1 0 -1
1 5 8 12 13	
5	
8 1 23 1 11	

В этом примере есть возрастающая последовательность из $a_0 = 1$, $a_1 = 5$, $a_2 = 8$, $a_3 = 12$ и $a_4 = 13$ длиной в $n = 5$ и пять чисел для поиска: 8 1 23 1 11. Видно, что $a_2 = 8$ и $a_0 = 1$, но чисел 23 и 11 нет в последовательности a , поэтому они имеют индекс -1. В итоге ответ: 2 0 -1 0 -1.

Листинг кода.

```
from create_arrays import *
from measurments import *

def get_data():
    with open('input.txt') as f:
        data = f.readlines()
```

```

    n, arr1, k, arr2 = int(data[0]), list(map(int, data[1].split())), int(data[2]),
list(map(int, data[3].split()))
    return n, arr1, k, arr2

@measure_base_performance
def binary_search(arr1, arr2):
    a, b = arr1, arr2
    n, k = len(a), len(b)
    inds = [-1] * k

    for i in range(k):
        low, high = 0, n - 1
        j = b[i]

        while low <= high:

            mid = low + ((high - low) >> 1)
            mid_val = a[mid]

            if mid_val < j:
                low = mid + 1

            elif mid_val > j:
                high = mid - 1

            else:
                inds[i] = mid
                break
    return inds

if __name__ == "__main__":
    generate_arrays(10**5, 10**5)
    ln, aarr, lk, barr = get_data()
    print('started algorithm')
    result = binary_search(aarr, barr)
    with open('output.txt', 'w') as fl:
        fl.write(" ".join(map(str, result)))
    check_arr = [aarr.index(i) if i in aarr else -1 for i in barr]

```

```
assert all([result[i] == check_arr[i] for i in range(lk)], 'wrong.....')
```

```
from create_arrays import *
from measurments import *

def get_data():
    with open('input.txt') as f:
        data = f.readlines()
        n, arr1, k, arr2 = int(data[0]), list(map(int, data[1].split())), int(data[2]),
list(map(int, data[3].split()))
        return n, arr1, k, arr2
@measure_base_performance
def binary_search(arr1, arr2):
    a, b = arr1, arr2
    n, k = len(a), len(b)
    inds = [-1] * k

    for i in range(k):
        low, high = 0, n - 1
        j = b[i]

        while low <= high:

            mid = low + ((high - low) >> 1)
            mid_val = a[mid]

            if mid_val < j:
                low = mid + 1

            elif mid_val > j:
                high = mid - 1

            else:
                inds[i] = mid
                break
    return inds
```

```

if __name__ == "__main__":
    generate_arrays(10**5, 10**5)
    ln, aarr, lk, barr = get_data()
    print('started algorithm')
    result = binary_search(aarr, barr)
    with open('output.txt', 'w') as fl:
        fl.write(" ".join(map(str, result)))
    check_arr = [aarr.index(i) if i in aarr else -1 for i in barr]
    assert all([result[i] == check_arr[i] for i in range(lk)], 'wrong.....')

```

Листинг кода для дополнительной функции

```

import numpy as np
import random
from merging_norm import *

def generate_arrays(n=None, k=None):
    if not n:
        n = random.randint(1, 10**5)
    if not k:
        k = random.randint(1, 10**5)
    arr_a = sorted(random.sample(range(1, 10**9), n))
    print('created a')
    arr_b = np.random.randint(1, 10**9 + 1, k)
    print('created b')
    fin_data = [f'{str(n)}\n', f'" ".join(map(str, arr_a))\n', f'{str(k)}\n', f'"
".join(map(str, arr_b))\n']

    with open('input.txt', 'w') as f:
        f.writelines(fin_data)
    return None

```

```

import numpy as np
import random
from merging_norm import *

def generate_arrays(n=None, k=None):
    if not n:
        n = random.randint(1, 10**5)
    if not k:

```

```

        k = random.randint(1, 10**5)
        arr_a = sorted(random.sample(range(1, 10**9), n))
        print('created a')
        arr_b = np.random.randint(1, 10**9 + 1, k)
        print('created b')
        fin_data = [f'{str(n)}\n', f'{" ".join(map(str, arr_a))}\n', f'{str(k)}\n', f'{"
".join(map(str, arr_b))}']

    with open('input.txt', 'w') as f:
        f.writelines(fin_data)
    return None

```

```

import os
import psutil
import time
import threading
from functools import wraps

_in_recursion = False

def measure_base_performance(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        global _in_recursion
        is_outer_call = not _in_recursion
        if is_outer_call:
            _in_recursion = True

        process = psutil.Process(os.getpid())
        peak = [process.memory_info().rss]
        running = [True]

        def monitor():
            while running[0]:
                mem = process.memory_info().rss
                if mem > peak[0]:
                    peak[0] = mem
                time.sleep(0.01)

        thread = threading.Thread(target=monitor)
        thread.start()
        result = func(*args, **kwargs)
        thread.join()
        return result

```

```

        t = threading.Thread(target=monitor)
        t.start()

        start_time = time.perf_counter()
        result = func(*args, **kwargs)
        end_time = time.perf_counter()

        running[0] = False
        t.join()

        peak_usage = peak[0] / 1024

        print(f'Function {func.__name__}\nTime taken: {end_time -
start_time:.4f} s\n'
              f'Peak memory: {peak_usage:.2f} KB')

        _in_recursion = False
        return result

    else:
        return func(*args, **kwargs)

return wrapper

```

```

import os
import psutil
import time
import threading
from functools import wraps

_in_recursion = False

def measure_base_performance(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        global _in_recursion
        is_outer_call = not _in_recursion
        if is_outer_call:
            _in_recursion = True

```

```

process = psutil.Process(os.getpid())
peak = [process.memory_info().rss]
running = [True]

def monitor():
    while running[0]:
        mem = process.memory_info().rss
        if mem > peak[0]:
            peak[0] = mem
        time.sleep(0.01)

t = threading.Thread(target=monitor)
t.start()

start_time = time.perf_counter()
result = func(*args, **kwargs)
end_time = time.perf_counter()

running[0] = False
t.join()

peak_usage = peak[0] / 1024

    print(f'Function {func.__name__}\nTime taken: {end_time -
start_time:.4f} s\n"
        f'Peak memory: {peak_usage:.2f} KB")

    _in_recursion = False
    return result

else:
    return func(*args, **kwargs)

return wrapper

```

Текстовое объяснение решения.

Алгоритм бинарного поиска строится на сравнении среднего элемента с искомым, делении массива на две части относительно среднего

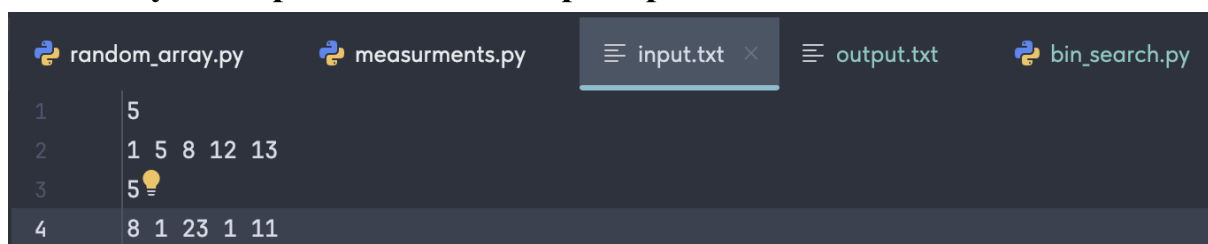
элемента, и поиска элемента в той части, относительно которой складывается отношение искомого и среднего элементов.

Функции передаются два массива – отсортированный и тот, элементы которого желают найти в первом. Инициализируются локальные переменные функции (для меньших затрат времени): *a*, *b* – для обозначения массивов, *n*, *k* – для обозначения длины массивов, *inds* – будущий список индексов, заранее заполнен -1, чтобы в случае отсутствия элемента в массиве *a* сократить время и количество операций.

Для каждого элемента (ищем по индексу) в массиве *b* по-новому запускается алгоритм поиска. Инициализируются переменные *low*, *high*, которые обозначают границы массива в данный момент рассмотрения, и фиксируется элемент $j = b[i]$. Пока границы не равны друг другу, производится поиск элемента: 1) ищется средний элемент массива $mid = low + ((high-low) >> 1)$; 2) *mid_value* сравнивается с *j*: если они равны, то в *inds* на позицию *i* записывается индекс *mid*; если $j < mid_value$, то берется левая половина массива и поиск рекурсивно продолжается там; если $j > mid_value$, то, наоборот, берется правая половина массива; 3) когда $low == high$, цикл прерывается и переходит к следующему элементу массива *b*. Функция возвращает массив найденных индексов.

Функция генерации массива была позаимствована из сортировки слиянием (принцип тот же, просто принимаются два аргумента, показывающих длины массивов, а массив *a* сортируется и создается с помощью *random.sample*). Для того чтобы уложиться в рамки 2 сек, было необходимо переписать декоратор, так как *tracemalloc* делал снимки памяти и фиксировал пики, что занимало достаточно много времени и тормозило алгоритм до 3 сек. В новом декораторе (актуален и для рекурсивных функций) с помощью модуля *psutil* фиксируется ID процесса (то есть выполняемой функции), а также фиксируются пики выделенной памяти каждый 0.01 сек, чтобы выявить реальный пик с помощью функции *monitor()*. Далее запускается отсчет времени и сама функция. Результаты выполнения выводятся в командную строку.

Результат работы кода на примерах из задания:



```
1 5
2 1 5 8 12 13
3 5
4 8 1 23 1 11
```

Рисунок 45 – Файл *input.txt* с входными данными из задания

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0000 сек	656 В
Пример из задачи	0.0000 сек	656 В
Сгенерированный пример	0.1761 сек	537 КВ
Верхняя граница диапазона значений входных данных из текста задачи	0.3126 сек	782 КВ

Вывод по задаче: Алгоритм бинарного поиска реализуется довольно просто и логично, сложность его $O(k \cdot \log n)$, он справляется даже с очень большими наборами значений. Декораторы занимают большую часть времени выполнения, что затрудняет возможность укладывания в рамки задания.

Задача №8. Умножение многочленов

Выдающийся немецкий математик Карл Фридрих Гаусс (1777—1855) заметил, что хотя формула для произведения двух комплексных чисел $(a + bi)(c + di) = ac - bd + (bc + ad)i$ содержит четыре умножения вещественных чисел, можно обойтись и тремя: вычислим ac , bd и $(a + b)(c + d)$ и воспользуемся тем, что $bc + ad = (a + b)(c + d) - ac - bd$.

Задача. Даны 2 многочлена порядка $n-1$: $a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \dots + a_1x + a_0$ и $b_{n-1}x^{n-1} + b_{n-2}x^{n-2} + \dots + b_1x + b_0$. Нужно получить произведение: $c_{2n-2}x^{2n-2} + c_{2n-3}x^{2n-3} + \dots + c_1x + c_0$, где:

$$c_{2n-2} = a_{n-1}b_{n-1}$$

$$c_{2n-3} = a_{n-1}b_{n-2} + a_{n-2}b_{n-1}$$

... ..

$$c_2 = a_2b_0 + a_1b_1 + a_0b_2$$

$$c_1 = a_1b_0 + a_0b_1$$

$$c_0 = a_0b_0$$

Пример. Входные данные: $n = 3$, $A = (3, 2, 5)$, $B = (5, 1, 2)$

$$A(x) = 3x^2 + 2x + 5$$

$$B(x) = 5x^2 + x + 2$$

$$A(x)B(x) = 15x^4 + 13x^3 + 33x^2 + 9x + 10$$

Ответ: $C = (15, 13, 33, 9, 10)$.

- Формат входного файла (input.txt). В первой строке число n - порядок многочленов A и B . Во второй строке коэффициенты многочлена A через пробел. В третьей строке коэффициенты многочлена B через пробел.

- Формат выходного файла (output.txt). Ответ - одна строка, коэффициенты многочлена $C(x) = A(x)B(x)$ через пробел.

- Нужно использовать метод "Разделяй и властвуй". Подсказка: любой многочлен $A(x)$ можно разделить на 2 части, например, $A(x) = 4x^3 + 3x^2 + 2x + 1$ разделим на $A_1 = 4x+3$ и $A_2 = 2x+1$. И многочлен $B(x) = x^3 + 2x^2 + 3x+4$ разделим на 2 части: $B_1 = x+2$, $B_2 = 3x+4$. Тогда произведение $C = A(x) * B(x) = (A_1B_1)x^n + (A_1B_2 + A_2B_1)x^{n/2} + A_2B_2$ - требуется 4 произведения (проверьте правильность данной формулы). Можно использовать формулу Гаусса и обойтись всего тремя произведениями.

Листинг кода

```
from polynome import *  
from measurments import *
```

```

def get_data():
    with open('input.txt') as f:
        data = f.readlines()
    n, pad = int(data[0]), int(data[1])
    a, b = list(map(int, data[2].split())), list(map(int, data[3].split()))
    return n, pad, a, b

```

@measure_performance

```

def poly_mult_4(p, a, b):
    if p == 1:
        return [a[0] * b[0]]
    if p > 2:
        a1 = a[:p//2]
        a2 = a[p//2:]
        b1 = b[:p // 2]
        b2 = b[p // 2:]

        c1 = poly_mult_4(p//2, a1, b1)
        c2 = poly_mult_4(p // 2, a2, b1)
        c3 = poly_mult_4(p // 2, a1, b2)
        c4 = poly_mult_4(p // 2, a2, b2)

        mid_sum = [x+y for x,y in zip(c2, c3)]

        fp = c1[:p//2]
        mp1 = [x+y for x, y in zip(c1[p//2:], mid_sum[: (p//2)-1])]
        me = [mid_sum[(p//2)-1]]
        mp2 = [x+y for x, y in zip(mid_sum[(p//2):], c4[: (p//2)-1])]
        lp = c4[(p//2)-1:]
        fin_arr = fp + mp1 + me + mp2 + lp
        return fin_arr
    return [a[0]*b[0], a[1]*b[0]+a[0]*b[1], a[1]*b[1]]

```

@measure_performance

```

def poly_mult_3(p, a, b):
    if p == 1:
        return [a[0] * b[0]]
    if p > 2:
        a1 = a[:p//2]

```

```

a2 = a[p//2:]
b1 = b[:p // 2]
b2 = b[p // 2:]

temp1 = [x + y for x, y in zip(a1, a2)]
temp2 = [x + y for x, y in zip(b1, b2)]
c1 = poly_mult_3(p//2, a1, b1)
c2 = poly_mult_3(p // 2, temp1, temp2)
c3 = poly_mult_3(p // 2, a2, b2)
c4 = [x - y - z for x, y, z in zip(c2, c1, c3)] #a1b2 + a2b1

fp = c1[:p//2]
mp1 = [x+y for x, y in zip(c1[p//2:], c4[:p//2-1])]
me = [c4[p//2-1]]
mp2 = [x+y for x, y in zip(c4[p//2:], c3[:p//2-1])]
lp = c3[p//2-1:]
fin_arr = fp + mp1 + me + mp2 + lp
return fin_arr

return [a[0]*b[0], a[1]*b[0]+a[0]*b[1], a[1]*b[1]]

if __name__ == "__main__":
    generate_coefs(100000)
    real_len, pad_len, coef_a, coef_b = get_data()
    zrs = (pad_len - real_len) * 2
    mult_coefs4 = poly_mult_4(pad_len, coef_a, coef_b)[zrs:]
    mult_coefs3 = poly_mult_3(pad_len, coef_a, coef_b)[zrs:]
    assert all([mult_coefs4[i] == mult_coefs3[i] for i in range(real_len)]),
'not equal.....'
    with open('output.txt', 'w') as fl:
        fl.write(" ".join(map(str, mult_coefs3)))
    print(mult_coefs3)

```

```

from polynome import *
from measurments import *

```

```

def get_data():
    with open('input.txt') as f:
        data = f.readlines()
    n, pad = int(data[0]), int(data[1])

```

```

a, b = list(map(int, data[2].split())), list(map(int, data[3].split()))
return n, pad, a, b

```

```

@measure_performance
def poly_mult_4(p, a, b):
    if p == 1:
        return [a[0] * b[0]]
    if p > 2:
        a1 = a[:p//2]
        a2 = a[p//2:]
        b1 = b[:p // 2]
        b2 = b[p // 2:]

        c1 = poly_mult_4(p//2, a1, b1)
        c2 = poly_mult_4(p // 2, a2, b1)
        c3 = poly_mult_4(p // 2, a1, b2)
        c4 = poly_mult_4(p // 2, a2, b2)

        mid_sum = [x+y for x,y in zip(c2, c3)]

        fp = c1[:p//2]
        mp1 = [x+y for x, y in zip(c1[p//2:], mid_sum[:(p//2)-1])]
        me = [mid_sum[(p//2)-1]]
        mp2 = [x+y for x, y in zip(mid_sum[(p//2):], c4[:(p//2)-1])]
        lp = c4[(p//2)-1:]
        fin_arr = fp + mp1 + me + mp2 + lp
        return fin_arr
    return [a[0]*b[0], a[1]*b[0]+a[0]*b[1], a[1]*b[1]]

```

```

@measure_performance
def poly_mult_3(p, a, b):
    if p == 1:
        return [a[0] * b[0]]
    if p > 2:
        a1 = a[:p//2]
        a2 = a[p//2:]
        b1 = b[:p // 2]
        b2 = b[p // 2:]

```

```

temp1 = [x + y for x, y in zip(a1, a2)]
temp2 = [x + y for x, y in zip(b1, b2)]
c1 = poly_mult_3(p//2, a1, b1)
c2 = poly_mult_3(p // 2, temp1, temp2)
c3 = poly_mult_3(p // 2, a2, b2)
c4 = [x - y - z for x, y, z in zip(c2, c1, c3)] #a1b2 + a2b1

fp = c1[:p//2]
mp1 = [x+y for x, y in zip(c1[p//2:], c4[:p//2-1])]
me = [c4[(p//2)-1]]
mp2 = [x+y for x, y in zip(c4[(p//2):], c3[:p//2-1])]
lp = c3[p//2-1:]
fin_arr = fp + mp1 + me + mp2 + lp
return fin_arr

return [a[0]*b[0], a[1]*b[0]+a[0]*b[1], a[1]*b[1]]

if __name__ == "__main__":
    generate_coefs(100000)
    real_len, pad_len, coef_a, coef_b = get_data()
    zrs = (pad_len - real_len) * 2
    mult_coefs4 = poly_mult_4(pad_len, coef_a, coef_b)[zrs:]
    mult_coefs3 = poly_mult_3(pad_len, coef_a, coef_b)[zrs:]
    assert all([mult_coefs4[i] == mult_coefs3[i] for i in range(real_len)]),
'not equal.....'
    with open('output.txt', 'w') as fl:
        fl.write(" ".join(map(str, mult_coefs3)))
    print(mult_coefs3)

```

Листинг кода функции для формирования коэффициентов полиномов

```

import random
import numpy as np
import math

def generate_coefs(n=None):
    if not n:
        n = random.randint(1, 25)

    if not np.log2(n).is_integer():

```

```

        k = math.ceil(np.log2(n))
        p = 2 ** k
        a = [0 for _ in range(p-n)] + [random.randint(-10**4, 10**4) for _ in
range(n)]
        b = [0 for _ in range(p - n)] + [random.randint(-10 ** 4, 10 ** 4) for
_ in range(n)]
    else:
        a = [random.randint(-10 ** 4, 10 ** 4) for _ in range(n)]
        b = [random.randint(-10 ** 4, 10 ** 4) for _ in range(n)]
        p = n
        all_data = [f'{n}\n', f'{p}\n', f'" ".join(map(str, a))\n', f'"
".join(map(str, b))}]

    with open('input.txt', 'w') as f:
        f.writelines(all_data)

```

```

import random
import numpy as np
import math

def generate_coefs(n=None):
    if not n:
        n = random.randint(1, 25)

    if not np.log2(n).is_integer():
        k = math.ceil(np.log2(n))
        p = 2 ** k
        a = [0 for _ in range(p-n)] + [random.randint(-10**4, 10**4) for _ in
range(n)]
        b = [0 for _ in range(p - n)] + [random.randint(-10 ** 4, 10 ** 4) for
_ in range(n)]
    else:
        a = [random.randint(-10 ** 4, 10 ** 4) for _ in range(n)]
        b = [random.randint(-10 ** 4, 10 ** 4) for _ in range(n)]
        p = n
        all_data = [f'{n}\n', f'{p}\n', f'" ".join(map(str, a))\n', f'"
".join(map(str, b))}]

    with open('input.txt', 'w') as f:

```

```
f.writelines(all_data)
```

Текстовое объяснение решения

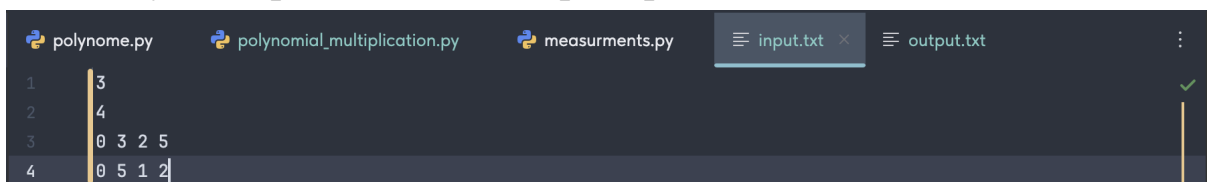
Для выполнения задания было создано несколько функций: `poly_mult_3()` для перемножения полиномов с помощью трех произведений, `generate_coefs()` для создания коэффициентов полиномов степени $n-1$.

Функция `generate_coefs()` работает по аналогии с функцией создания матриц для метода Штрассена: здесь также необходимы полиномы, количество слагаемых которого является степенью двойки. Иначе, если n передано/сформировано не как степень двойки, находится ближайшая степень двойки, превосходящая n , спереди коэффициенты дополняются нулями.

По формулам из задания в функции `poly_mult_3()`, которой передается дополненная длина полиномов и сами полиномы (их коэффициенты), коэффициенты полиномов делятся пополам (левая и правая половины массивов) до того момента, как их длина не достигнет 1. Если длина достигает 1, то возвращается просто произведение двух чисел. Далее начинается восстановление полинома. По формулам находится коэффициент $ac + bd$, который отражает средние коэффициенты полинома. Итоговые коэффициенты формируются следующим образом: первая часть коэффициентов – часть массива, образованного произведением $a_1 * b_1$, до элемента с индексом $p//2$; средняя часть коэффициентов формируется как сумма оставшейся части массива c_1 и часть массива $ac + bd$ той же длины, отсчитывая с начала; средний элемент массива $ac + bd$ ни с чем не суммируется и остается тем же; продолжение средней части массива формируется как часть массива $ac + bd$, начиная с индекса $p//2$, просуммированная с частью массива c_3 до элемента с индексом $p//2 - 1$; оставшаяся часть массива c_3 остается неизменной. Функция возвращает массив коэффициентов.

Если добавленная длина массива полиномов больше изначальной, то массив обрезается (чтобы не было 0).

Результат работы кода на примерах из задания:



```
polynome.py polynomial_multiplication.py measurments.py input.txt x output.txt
1 3
2 4
3 0 3 2 5
4 0 5 1 2
```

Рисунок 47 – Файл `input.txt` с входными данными из примера 1


```
polynome.py polynomial_multiplication.py measurments.py input.txt x output.txt
```

```
1 1
2 1
3 -6619
4 -241
```

polynome.py polynomial_multiplication.py measurments.py input.txt output.txt

```
polynome.py polynomial_multiplication.py mesurments.py input.txt output.txt
```

```
1 1000  
2 1024  
3 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 -9842 -8587 1421 6786 2482 9583 -2084 5816 -89 5142  
3731 6356 -4242 7179 -2140 1899 7889 -1532 3888 -4867 -8279 3050 5548 -4375 9958 1772 -811 -3212 8876  
-7569 3725 -4195 360 -7486 -8687 9364 -52 4688 6164 -2327 -958 7456 -3931 -4737 -3022 -4846 8840 5430  
9922 368 2764 -2501 6651 3403 -8745 -3247 -740 -2651 -2406 832 -200 7704 -7697 -7230 -7213 3564 -9969  
7059 7715 5021 2825 4439 3846 5058 1187 -8794 -2263 6054 3882 -3685 9991 8776 -7112 3936 6739 3563  
-6820 -5267 9430 4792 4280 -5593 2532 -5648 -2900 9585 5562 -259 -6777 -5156 -617 -7108 7750 -2759  
-4935 9381 -652 8440 -9489 4798 110 -505 4948 -9284 -1054 -6100 3786 7320 -9183 8429 6307 5598 -9211  
-154 -9307 -2943 -1548 -9628 -9833 7305 5981 -2973 -5224 -6279 -4522 6928 8673 -8231 -5136 -8401 9612  
-6736 3776 -7996 -5463 -7682 -7197 7011 -6193 -5966 -3631 775 6194 -8246 -5053 -8881 -6852 3504 -2996  
3255 -3074 -8943 -8437 306 7592 7817 -6405 -6329 -4497 -7636 4785 -5283 7606 -8256 5665 8919 -3694  
-8015 1805 9749 -3852 -4716 6059 5122 455 -8985 6933 2616 -3387 5617 -8409 -5314 7453 5841 6149 2075  
6640 -6693 -7929 -5712 -9063 4417 -3366 -8626 -9986 -7819 -4693 -4164 2368 -5160 4444 1546 -6038 -959  
7730 9399 3560 1813 2548 7662 -987 -5878 5583 5518 -2379 -1404 5601 8823 -4384 -7201 5749 9185 4641
```

polynome.py polynomial_multiplication.py measurements.py input.txt output.txt

```

1 -32016026 27053743 -38430576 -123678018 -80644560 9277371 43232335 267527125 42639024 22202215 49470987
  12915124 105657855 195520898 65103272 206855499 18472671 -43050511 3889239 -181011227 -255184495
-109310895 -365907394 88380761 -141553013 -128134707 -140463256 -116053140 -183917215 -156081738
-144484594 -106962700 -221559499 -110419483 -199634665 -5409889 -4408406 -29897364 -95953321 -71710242
  259356693 -197229584 310649397 68844283 131314371 -58038648 -12307122 -282806495 116700525 -280300234
  290083286 -117440166 -337976373 -64413818 -271144842 192310604 153077672 -409291954 217292604
-413681670 -127911884 230510884 -259095952 -112818490 -223063848 -107770277 -208575108 5829755
-303211062 93093628 349427769 194646615 251630617 474099524 -178376487 138707039 -15138845 200726499
-257422865 76800405 85462372 393145019 -181183129 -117517128 -552125870 192428255 -6216073 -524370800
359948928 -195371962 -312895707 -696313982 -76122103 301807986 -355076356 -117769629 -380546539
4011237 199260645 -84012484 139119388 -343653396 49881060 -262727573 555137169 2740712 -94341180
164802063 142264034 -121099180 198227083 -988392925 -425062714 -157551448 600277222 -459797964
13022116 -156195741 45707637 -241764791 -125849254 16304817 -126322852 -285530402 -369687061
1012008445 -682221579 70327254 211665046 420718024 -243248682 -446028880 324567316 -83288325
-487120400 539960991 -272091511 631441952 11649414 558777686 237179145 359738574 -211736191 -322537542

```

Таблица затрат памяти и времени по заданию 10:

Нижняя граница диапазона значений входных данных из текста задачи	0.0000 сек	0.09 KB
Пример 1	0.0001 сек	0.50 KB
Пример 2	0.0001 сек	0.50 KB
Сгенерированный пример	0.8832сек	272.87 KB
Верхняя граница диапазона значений входных данных из текста задачи	0.9498 сек	295.47 KB

Вывод по задаче: В функции умножения полиномов использован прием из бинарного поиска: деление массива на две части и рекурсивное повторение процедуры до того момента, пока не достигается длина 1. Использование метода Гаусса значительно ускоряет алгоритм, однако глубокая рекурсия с большим количеством умножений и большими по модулю числами все равно сказывается на работе алгоритма после 1000 значений в коэффициентах.

Вывод

Алгоритмы сортировки с принципом “разделяй и властвуй” наиболее эффективны по сравнению с алгоритмами со сложностью $O(n^2)$. С помощью этого принципа работу алгоритмов получается оптимизировать, сделать более надежной и быстрой. Этот принцип можно применять не только в алгоритмах сортировки, но и для вычисления матриц, а также произведений полиномов.